A near-linear-time randomized cost approximation for (1,2)-TSP

Soheil Behnezhad, Mohammad Roghani, Aviad Rubinstein, Amin Saberi

arxiv:2301.05350v2; CC BY 4.0

Editorial note. The single primary target is thm: 12tsp. Whole author mathematical body and support retained; inputs/includes and author bbl mechanically expanded. Only text-font and XeLaTeX wrapping adapted; native original files preserved.

Allowed standard prerequisites (editorial).

- Standard algorithmic and mathematical background cited by the authors. All source-local constructions, proofs, appendices and references are retained; no new mathematical assumptions added.

Abstract

We study sublinear time algorithms for the traveling salesman problem (TSP). First, we focus on the closely related maximum path cover problem, which asks for a collection of vertex disjoint paths that include the maximum number of edges. We show that for any fixed $\varepsilon > 0$, there is an algorithm that $(1/2 - \varepsilon)$-approximates the maximum path cover size of an n-vertex graph in $\widetilde{O}(n)$ time. This improves upon a $(3/8 - \varepsilon)$-approximate $\widetilde{O}(n\sqrt{n})$-time algorithm of Chen, Kannan, and Khanna [ICALP'20].

Equipped with our path cover algorithm, we give an $\widetilde{O}(n)$ time algorithm that estimates the cost of $(1, 2)$-TSP within a factor of $(1.5 + \varepsilon)$ which is an improvement over a folklore $(1.75 + \varepsilon)$-approximate $\widetilde{O}(n)$-time algorithm, as well as a $(1.625 + \varepsilon)$-approximate $\widetilde{O}(n\sqrt{n})$-time algorithm of [CHK ICALP'20]. For graphic TSP, we present an $\widetilde{O}(n)$ algorithm that estimates the cost of graphic TSP within a factor of 1.83 which is an improvement over a 1.92-approximate $\widetilde{O}(n)$ time algorithm due to [CHK ICALP'20, Behnezhad FOCS'21]. We show that the approximation can be further improved to 1.66 using $n^{2-\Omega(1)}$ time.

All of our $\widetilde{O}(n)$ time algorithms are information-theoretically time-optimal up to $\operatorname{poly}\log n$ factors. Additionally, we show that our approximation guarantees for path cover and $(1, 2)$-TSP hit a natural barrier: We show better approximations require better sublinear time algorithms for the well-studied maximum matching problem.

Contents

1 Introduction 1

2 Technical Overview 3

3 Preliminaries 5

4 New Meta Algorithms for Maximum Path Cover 6

5 A Local Query Process for Algorithm 2 and its Complexity 9

6 Our Estimator for Maximum Path Cover 16

7 Our Estimator for (1,2)-TSP 19

8 Our Estimator for Graphic TSP 21

9 Further Improvement for Graphic TSP 24

10 A Slightly Subquadratic Algorithm for Graphic TSP 28

11 Lower Bound for Approximating Maximum Path Cover 29

11.1 "Conditional" Hardness for the Approximation Ratio 29

11.2 Information-Theoretic Lower Bounds on the Running Time 31

A Implementation Details 35

1 Introduction

The traveling salesman problem (TSP) is a central problem in combinatorial optimization. Given a set V of n vertices and their pairwise distances, it asks for a Hamiltonian cycle of the minimum cost. In this paper, we study sublinear time algorithms for TSP. The algorithm is given query access to the distance pairs, and the goal is to estimate the solution cost in time sublinear in the input size (which is $\Theta(n^2)$).

TSP is NP-hard to approximate within a polynomial factor for an arbitrary distance function. As such, much of the work in the literature has been on more specific distance functions. Some notable examples include graphic TSP [16, 23, 24, 28, 11] where the distances are the shortest paths over an arbitrary unweighted undirected graph, $(1,2)$-TSP [1, 11, 7, 19, 22] where the distances are 1 or 2, and more generally metric TSP [18, 15, 13, 29] where the distances satsify triangle inequality.

In 2003, Czumaj and Sohler [14, 15] showed that for any fixed $\varepsilon > 0$, a $(1+\varepsilon)$-approximation of the cost of metric minimum spanning tree (MST) and thus a $(2+\varepsilon)$-approximation of the cost of metric TSP can be found in $\widetilde{O}(n)$ time. Twenty years later, it still remains a major open problem to either break two-approximation in $n^{2-\Omega(1)}$ time or prove a lower bound.[1] However, better bounds are known for both graphic TSP and $(1,2)$-TSP. In this paper, we present improved algorithms for these two well-studied variants of TSP. Our main tool to achieve this is an improved algorithm for the closely related maximum path cover problem which might be of independent interest.

Maximum Path Cover: The maximum path cover in a graph is a collection of vertex disjoint paths with the maximum number of edges in it. The (almost) 1/2-approximate maximum matching size estimator of Behnezhad [2] immediately implies an (almost) 1/4-approximation for the maximum path cover problem in $\widetilde{O}(n)$ time.[2] This can be improved to an (almost) $(3/8 = .375)$-approximation using the matching-pair idea of Chen, Kannan, and Khanna [11] in $\widetilde{O}(n\sqrt{n})$-time.[3] Our first main contribution is an improvement over both of these results:

Result 1 (Formally as Theorem 6.8). For any $\varepsilon > 0$, there is a randomized algorithm that w.h.p. $(1/2-\varepsilon)$-approximates the size of maximum path cover in $\widetilde{O}(n \cdot \mathrm{poly}(1/\varepsilon))$ time.

Besides quantitavely improving prior work both in the running time and the approximation ratio, Result 1 reaches a qualitatively important milestone as well. First, the running time of Result 1 is information-theoretically optimal up to $\mathrm{poly}\log n$ factors (the lower bound holds for any constant approximation — see Section 11.2). Second, its approximation ratio hits a rather important barrier. We give a non-trivial reduction that shows a $(1/2+\Omega(1))$-approximation in $\widetilde{O}(n)$ time for maximum path cover would imply the same bound for maximum matching in bipartite graphs. Such a result has remained elusive for matching, which is one of the most extensively studied problems in the literature of sublinear time algorithms. See Section 11.

It is also worth noting that in bounding the running time of our algorithm in Result 1, we use connections to parallel algorithms. Such a connection was previously only used for matchings [2].

[1] See e.g. Open Problem 71 on sublinear.info [17].

[2] The application of sublinear time maximum matching algorithms for approximating maximum path cover was first proposed by Gupta and Onak. See [17].

[3] We note that even though a subsequent result of Behnezhad [2] improved the running time for maximal matchings and graphic TSP from $O(n\sqrt{n})$ in [11] to $\widetilde{O}(n)$, it is not immediately clear whether the same holds for path cover and $(1,2)$-TSP as they rely on a different notion of a matching pair.

Running Time	Approximation Ratio	Metric	Reference
$\widetilde{O}(n)$	$1.75+\varepsilon$	(1,2)	Folklore
$\widetilde{O}(n\sqrt{n})$	$1.625+\varepsilon$	(1,2)	Chen, Kannan, and Khanna [11]
$\widetilde{O}(n)$	$1.5+\varepsilon$	(1,2)	This work (Result 2)
$\widetilde{O}(n)$	1.929	Graphic	Chen, Kannan, and Khanna [11]
$\widetilde{O}(n)$	1.834	Graphic	This work (Result 3)
$n^{2-\Omega(1)}$	1.667	Graphic	This work (Result 4)
$\Omega(n^2)$	$1+\varepsilon$	(1,2) & Graphic	Chen, Kannan, and Khanna [11]
$n^{1+\Omega(1)}$ (Conditional)	$1.5-\varepsilon$	(1,2) & Graphic	This work (Theorem 11.5)

Table 1: Comparison of running time and approximation ratio of our TSP algorithms and lower bounds with prior work.

$(1,2)$-TSP: The $(1,2)$-TSP problem has been studied extensively in the classical setting. In his landmark paper, Karp [19] showed that $(1,2)$-TSP is NP-hard. Papadimitriou and Yannakakis [26] then proved its APX-hardness. Since then there has been a significant amount of work on $(1,2)$-TSP in the classical setting. The current best known inapproximability bound for $(1,2)$-TSP is 535/534 [20]. After a series of works, the best known polynomial time approximation is 8/7 [7] which can be implemented in $O(n^3)$ time [1]. For sublinear time algorithms, an $\widetilde{O}(n)$-time (almost) 1.75-approximation is folklore [17]. Chen, Kannan, and Khanna [11] improved the approximation to (almost) 1.625 in $\widetilde{O}(n\sqrt{n})$ time.

It is not hard to see that up to a small additive error of 1, $(1,2)$-TSP is equivalent to finding a maximum path cover on the weight-1 edges and then connecting their endpoints via weight-2 edges. A simple calculation shows that any α-approximation for the maximum path cover problem leads to a $(2-\alpha)$-approximation for $(1,2)$-TSP. Our path cover algorithm of Result 1 immediately implies the following result as a corollary:

Result 2. For any $\varepsilon > 0$, there is a randomized algorithm that w.h.p. $(1.5+\varepsilon)$-approximates the cost of $(1,2)$-TSP in $\widetilde{O}(n \cdot \mathrm{poly}(1/\varepsilon))$ time.

Similar to Result 1, the running time of Result 2 is information-theoretically optimal up to $\mathrm{poly}\log n$ factors, and its approximation ratio hits a natural barrier due to a connection to sublinear time matching that we establish in this work.

Graphic TSP: The graphic TSP problem is equivalent to finding a tour of the minimium size that visits all the vertices. This is an important instance of TSP that has received a lot of attention over the years. For polynomial time algorithms, a 1.5-approximation of Christofides [13] (which also works more generally for metric TSP) had remained the best known until a series of works over the last decade improved it to $(1.5-\varepsilon_0)$ [16], 1.461 [23], 1.444 [24], and finally to 1.4 [28]. For sublinear time algorithms, Chen, Kannan, and Khanna [11] showed that an (almost) $(27/14 \approx 1.928)$-approximation of graphic TSP can be obtained in $\widetilde{O}(n\sqrt{n})$ time. The running time was subsequently improved to $\widetilde{O}(n)$ by Behnezhad [2].

We first show that plugging Result 1 into the framework of [11] immediately improves their approximation from 1.928 to (almost) 1.9 while keeping the running time $\widetilde{O}(n)$. We then give a more fine tuned algorithm that obtains a much improved approximation ratio of $(11/6 \approx 1.833)$.

Result 3. For any $\varepsilon > 0$, there is a randomized algorithm that w.h.p. $(1+\varepsilon)(\frac{11}{6} \approx 1.833)$-approximates the cost of graphic TSP in $\widetilde{O}(n \cdot \text{poly}(1/\varepsilon))$ time.

Over the past few years, significant advancements have been made in the development of sublinear matching algorithms. Several recent results [3, 4, 6, 5, 8, 9] have led to the creation of a $(1, \varepsilon n)$-approximation algorithm for maximum matching, with running time of $n^{2-\Omega_\varepsilon(n)}$. Leveraging these sublinear algorithms, we have devised a slightly subquadratic algorithm that provides a more accurate estimation of the size of graphic TSP.

Result 4. For any $\varepsilon > 0$, there is a randomized algorithm that w.h.p. $(1+\varepsilon)(\frac{5}{3} \approx 1.666)$-approximates the cost of graphic TSP in $n^{2-\Omega_\varepsilon(1)}$ time.

We contrast our results with prior sublinear TSP algorithms in Table 1.

Further related work: Finally, we note that in a recent paper, Chen, Khanna, and Tan [12] show that assuming that the metric has a spanning tree supported on weight 1 edges, one can obtain a $(2-\varepsilon_0)$-approximation with $\widetilde{O}(n\sqrt{n})$ queries for some small unspecified constant $\varepsilon_0 > 0$. While this is a more general metric than graphic TSP and (1,2)-TSP that we study in this paper, we note that the two papers are orthogonal and their results are incomparable. In particular, the techniques developed in this paper are specifically designed to improve the approximation to much below 2, whereas [12] focuses on generalizing the distance function while beating 2.

2 Technical Overview

In this section, we give an overview of our algorithms, especially our sublinear time maximum path cover algorithm of Result 1 which is the key to the other results as well.

Let us start with using matchings to approximate maximum path cover. Consider a graph that has a Hamiltonian path. Here, the optimal maximum path cover has size $n-1$. On the other hand, any maximum matching can have at most $n/2$ edges, which is by a factor 2 smaller than our optimal path cover. On top of this, we only know close to $1/2$ approximations for maximum matching if we restrict the running to be close to linear in n [2, 5], thus can only achieve an approximation close to $1/4$.

Instead of a single matching, Chen, Kannan, and Khanna [11] showed how to estimate the number of edges in a maximal matching pair in $\widetilde{O}(n\sqrt{n})$ time, where a matching pair is simply two edge disjoint matchings. It is not hard to see that the number of edges in a maximal matching pair is at least half the number of edges in a maximum path cover. The problem, however, is that a maximal matching pair is not a collection of paths! In particular, the two matchings can form cycles of length as small as four. Therefore, one may only be able to use $3/4$ fraction of the edges of a matching pair in a path cover. This is precisely why the algorithm of [11] only obtains a $\frac{1}{2} \times \frac{3}{4} = \frac{3}{8}$ approximation for path cover, and a $2 - \frac{3}{8} = 1.625$ approximation for $(1,2)$-TSP.

If we could modify the matching pair algorithm of [11], and avoid cycles by manually excluding

edges whose endpoints are the endpoints of a path in the current matching pair, then we could avoid the 3/4 factor loss discussed above and achieve a 1/2-approximation. Unfortunately, checking whether the endpoints of an edge are endpoints of a path requires knowledge about whether a series of other edges belong to the solution, which seems hard to implement in sublinear time.

Instead of checking for cycles manually, we introduce the following Algorithm 1 which avoids cycles more naturally. While our final algorithm is a modified variant of Algorithm 1 described below, we start with Algorithm 1 as we believe it provides the right intuition.

Algorithm 1: A new algorithm for path cover.

---

1 Initialize $P \leftarrow \emptyset$.

2 Each vertex v has two ports that we denote by v^0 and v^1. Each of these ports throughout the algorithm will be either free or occupied. Initially, all ports are free.

3 Iterate over the edges in some ordering π. Upon visiting an edge $e = (u, v)$:

- If v^0 and u^0 are free, add e to P, mark v^0 and u^0 as occupied, and skip to the next edge.
- If v^1 and u^0 are free, add e to P, mark v^1 and u^0 as occupied, and skip to the next edge.
- If v^0 and u^1 are free, add e to P, mark v^0 and u^1 as occupied, and skip to the next edge.

Return P.

Two properties of Algorithm 1 are crucial. First, it prioritizes occupying (u^0, v^0) (compared to (u^1, v^0) or (u^0, v^1)) which in particular implies that any component in P must have a (u^0, v^0) edge. Second, it never occupies (u^1, v^1) with an edge (u, v). While it is easy to see that the output of Algorithm 1 has maximum degree 2, and is thus a collection of paths or cycles, the two properties above actually guarantee that it never includes any cycle. See Figure 1. We provide the formal proof of this later in Section 4. Additionally, we show that the output of Algorithm 1 must be at least half the size of a maximum path cover, as we prove next. Hence, if we manage to estimate the size of the output P of Algorithm 1, then we have proved Result 1.

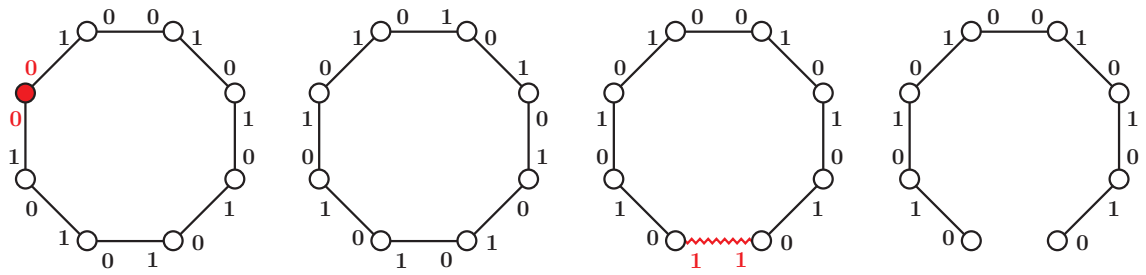


Figure 1: Examples of why the output of Algorithm 1 will not have cycles.

Our final algorithm is slightly different from Algorithm 1 discussed above. In particular, we slightly relax it—see Algorithm 2—so that it can be solved via a randomized greedy maximal independent set (RGMIS), for which we have a rich toolkit of sublinear time estimators. Existing approaches (particularly the algorithm of Yoshida, Yamamoto, and Ito [30] and its two-step implementation by Chen, Kannan, and Khanna [11]) can be employed to estimate the value of this modified Algorithm 2 in $\widetilde{O}(n\sqrt{n})$ time. We achieve the improved, and near tight, $\widetilde{O}(n)$ time bound guarantee of Result 1 by building on the analysis of Behnezhad [2] for maximal independent set on the line graphs (i.e., maximal matchings). Though we note that several new ideas are needed,

because the MIS graph in our case will not be exactly a line graph. We defer more discussions about this to Sections 4 and 5.

Implications for TSP: By having an α-approximate maximum path cover algorithm, we immediately obtain a $(2-\alpha)$-approximation for $(1,2)$-TSP. Therefore, the algorithm above immediately proves Result 2 that we can (almost) 1.5-approximate $(1,2)$-TSP in $\widetilde{O}(n)$ time. For our Result 3 for graphic TSP, we first observe that our improved path cover algorithm can be employed to provide a better lower bound for the optimal TSP solution. This improves the 1.92-approximation of [11] as black-box to 1.9-approximation (Section 8). However, the final improvement to 1.83 requires more ideas, in particular, on how to better estimate the number of certain bridges in the graph. See Section 9 for more details about this.

3 Preliminaries

Problem Definition: In the sublinear TSP problem, we have a set V of n vertices and a distance function $d : V \times V \to \mathbb{R}_+$. The algorithm has query access to this distance function. Namely, for any pair (u,v) of the vertices of its choice, the algorithm may query the value of $d(u,v)$. The goal is to design an algorithm that runs in sublinear time in the input size, which is $\Theta(n^2)$ (all the distance pairs), and produces an estimate of the size of the optimal TSP solution. Denoting the optimal TSP value by $\tau(V)$, we say an estimate $\widetilde{\tau}(V)$ provides an α-approximation for $\alpha \geq 1$ if

$$\tau(V) \leq \widetilde{\tau}(V) \leq \alpha \cdot \tau(V).$$

We focus specifically on graph TSP and $(1,2)$-TSP problems. In graphic TSP, the distance function d is the shortest path metric on an unweighted undirected graph G that is unknown to the algorithm. Note, however, that the distance queries essentially provide adjacency matrix access to this graph G. In $(1,2)$-TSP, the assumption is that $d(u,v) \in \{1,2\}$ for every pair u,v. In the case of $(1,2)$-TSP we may use G to refer to the subgraph induced on the pairs with distance 1.

Defining graph G as above, we use n to denote the number of its vertices, m to denote the number of its edges, Δ to denote its maximum degree, $\mu(G)$ to denote its maximum matching size, $\nu(G)$ to denote its minimum vertex cover size, and $\bar{d}$ to denote its average degree.

Path Cover Definitions: Given an unweighted graph G, a path cover in G is a collection of vertex disjoint paths in G. A maximum path cover is a path cover of G with the maximum number of edges in it (note that we are not counting the number of paths, but rather the total number of edges in them). We use $\rho(G)$ to denote the size of the maximum path cover in G. We say an estimate $\widetilde{\rho}(G)$ for $\rho(G)$ provides an (α,ε)-approximation for $\alpha,\varepsilon \in [0,1]$ if

$$\alpha \cdot \rho(G) - \varepsilon n \leq \widetilde{\rho}(G) \leq \rho(G).$$

We may also use α-approximation instead of $(\alpha,0)$-approximation.

Graph Theory Definitions/Tools: A bridge (cut edge) in a graph is an edge whose deletion increases the number of connected components. Similarly, a cut vertex is a vertex whose deletion (along with its edges) increases the number of connected components. A biconnected graph is a connected graph with no cut vertex. Also, a biconnected component (block) of a graph is a maximal biconnected subgraph of the original graph. A non-trivial biconnected component is a block that is not a

bridge. We say a graph is 2-edge-connected if there is no bridge in the graph. A 2-edge-connected component of a graph is maximal 2-edge-connected subgraph of the original graph. The bridge-block tree of a graph is a tree obtained by contracting the 2-edge-connected components; note that the edge set of a bridge-block tree correspond to the bridges in the original graph.

We use the following classic theorem of König [21] that the size of the minimum vertex cover is equal to the size of maximum matching in bipartite graphs. Namely:

Proposition 3.1 (König's Theorem). In any bipartite graph G, $\mu(G) = \nu(G)$.

Probabilistic Tools: In our proofs, we use the following standard concentration inequalities.

Proposition 3.2 (Chernoff Bound). Let $X_1, X_2, \ldots, X_n$ be independent Bernoulli random variables. Let $X = \sum_{i=1}^{n} X_i$. For any $t > 0$, $\Pr[|X - \mathrm{E}[X]| \geq t] \leq 2\exp\left(-\frac{t^2}{3\,\mathrm{E}[X]}\right)$.

Proposition 3.3 (Hoeffding's Inequality). Let $X_1, X_2, \ldots, X_n$ be independent random variables such that $a \leq X_i \leq b$. Let $\bar{X} = (\sum_{i=1}^{n} X_i)/n$. For any $t > 0$, $\Pr[|\bar{X} - \mathrm{E}[X]| \geq t] \leq 2\exp\left(-\frac{2nt}{(b-a)^2}\right)$.

4 New Meta Algorithms for Maximum Path Cover

In this section, we present a new meta algorithm for maximum path cover that obtains a 1/2-approximation. The algorithm, as we will state it in this section, will not be particularly in the sublinear time model. We discuss its sublinear time implementation later in Sections 5 and 6.

Our starting point is the Algorithm 1 described in Section 2. Let us first formally prove that it obtains a 1/2-approximation, and that no component in it is a cycle.

Claim 4.1. The output of Algorithm 1 is a collection of disjoint paths.

Proof. Since P has maximum degree two, it suffices to show none of its connected components are cycles. Property (i) above implies that at any point during the algorithm, any degree one vertex v has its port v^0 occupied. Now take an edge $e = (u, v)$ that forms a cycle if added to P. Both u and v must have degree one and so u^0 and v^0 are occupied. Since by property (ii) edge e does not occupy both v^1 and u^1, the algorithm does not add e to P thus not completing a cycle. □

Claim 4.2. Let $P^\star$ be any path cover using weight one edges. Then the output of Algorithm 1 has size at least $\frac{1}{2}|P^\star|$.

Proof. For any edge $e = (u, v) \in P^\star$ define $\phi(e) = \frac{1}{4}(\deg_P(u) + \deg_P(v))$. We first claim that for every edge $e = (u, v)$ in G, we have $\phi(e) \geq 1/2$ (or, equivalently, $\deg_P(u) + \deg_P(v) \geq 2$). This is clear for edges $e \in P$ due to the contribution of e itself to its endpoints' degrees, so fix $e \notin P$. Consider the time that we process $e = (u, v)$ in the algorithm and decide not to add it to P. We claim that out of v^0, v^1, u^0, u^1 at least two ports must be occupied. Suppose w.l.o.g. and for contradiction that only v^x is occupied for $x \in \{0, 1\}$. Then (u, v) can occupy v^{1-x} and u^x and be added to P. This contradicts (u, v) not being added to P and proves our claim that $\phi(e) \geq 1/2$.

From the discussion above, we get that

$$\sum_{e \in P^\star} \phi(e) \geq \sum_{e \in P^\star} 1/2 = |P^\star|/2.$$

Moreover, because every vertex has degree at most two in $P^\star$, we get

$$\sum_{e \in P^\star} \phi(e) = \frac{1}{4} \sum_{(u,v) \in P^\star} \deg_P(u) + \deg_P(v) \leq \frac{1}{4} \cdot 2 \sum_{v \in V} \deg_P(v) = |P|.$$

The two inequalities above combined imply that $|P| \geq |P^\star|/2$. □

As discussed, our final algorithm is different from Algorithm 1 discussed above. One problem with Algorithm 1 is that it cannot be cast as an instance of the randomized greedy maximal independent set (RGMIS) algorithm for which there is a rich toolkit of sublinear time estimators. To remedy this, we present a modified variant of Algorithm 1 whose output is (almost) as good, but in addition can be modeled as an instance of RGMIS. We denote the output of RGMIS on a graph G with a permutation π on its vertices by $\text{RGMIS}(G, \pi)$.

The algorithm is stated below as Algorithm 2. Similar to the output of Algorithm 1, the output of Algorithm 2 can be verified to have maximum degree two. Thus, it is a collection of paths and cycles. But unlike Algorithm 1, the output of Algorithm 2 can have cycles. This happens since, unlike Algorithm 1, each connected component of the output of Algorithm 2 is not guaranteed to have an edge (u, v) occupying both u^0 and v^0. Nonetheless, we are able to show that this bad event only happens for a small fraction of connected components of the output of Algorithm 2 in expectation, and so once we remove one edge of each of these cycles, the resulting collection of disjoint paths has almost the same size.

Algorithm 2: A modification of Algorithm 1 that uses RGMIS.

1 Parameter: K (think of it as a large constant integer).
2 Let $G = (V, E)$ be the subgraph of weight one edges. We construct a graph $H = (V_H, E_H)$ from G on which we run RGMIS.
3 Each vertex in H corresponds to an edge e in G and two ports (as in Algorithm 1) of the endpoints of e that it occupies. Formally, for any $(u, v) \in E$ we have $K + 2$ vertices in H:

- One vertex that corresponds to occuping u^0 and v^1.
- One vertex that corresponds to occuping u^1 and v^0.
- K vertices that each corresponds to occuping u^0 and v^0.

Consider two distinct vertices a and b in H corresponding to edges e_a and e_b in G:

- If $e_a = e_b$ then we add an edge between a and b in H.
- If e_a and e_b share exactly one endpoint v and both a and b occupy the same port of v, we add an edge between a and b in H.

Find a randomized greedy maximal independent set I of H.
Let P be the set of edges in G corresponding to the vertices in I.
Return P.

Observation 4.3. Let C be a connected component in the output of Algorithm 2. If C is a cycle, then every edge in C occupies one 0-port and one 1-port (that is, no edge occupies two 0-ports).

Proof. Suppose that C has n' vertices. Since each vertex in a cycle has degree two, both ports of each vertex in C must be occupied. Hence, n' 0-ports and n' 1-ports of C are occupied in total. Given that any edge occupies at least one 0-port by the algorithm, we cannot have an edge that occupies two 0-ports, or else we should occupy more 0-ports than 1-ports of C, which is a contradiction. □

Next, we show that up to a factor of $(1+2/k)$ which is negligible for K in the order $1/\varepsilon$, the output of Algorithm 2 is an (almost) 1/2-approximation of the maximum path cover value.

Observation 4.4. Let C be a connected component in the output of Algorithm 2. If C is a path, then it contains at most one edge that occupies two 0-ports.

Proof. Let C be the path $(v_1, v_2, \ldots, v_r)$. Since the degree of any vertex v_i for $1 < i < r$ is two in the path, both ports of v_i must be occupied. For v_1 and v_r, on the other hand, only one port is occupied. Hence, the total number of 0-ports that are occupied by C minus the number of 1-ports occupied by it is at most two. This means that there is at most one edge that occupies two 0-ports since all other types of edges occupy exactly one 0-port and one 1-port. □

Lemma 4.5. let P be the output of Algorithm 2 on graph G. Then

$$\frac{1}{2}\rho(G) \leq \mathrm{E}\,|P| \leq \left(1 + \frac{2}{K}\right)\rho(G),$$

where the expectation is taken over the randomization of computing RGMIS in Algorithm 2.

Proof. Let P^* be a maximum path cover. For any edge $e = (u, v) \in P^\star$ define $\phi(e) = \frac{1}{4}(\deg_P(u) + \deg_P(v))$. With the exact same argument as in the proof of Claim 4.2, we get that $\phi(e) \geq 1/2$, which implies

$$\sum_{e \in P^\star} \phi(e) \geq \sum_{e \in P^\star} 1/2 = \rho(G)/2.$$

Since the degree of each vertex in P is at most two, we get

$$\sum_{e \in P^\star} \phi(e) = \frac{1}{4} \sum_{(u,v) \in P^\star} \deg_P(u) + \deg_P(v) \leq \frac{1}{4} \cdot 2 \sum_{v \in V} \deg_P(v) = |P|.$$

By combining above inequalities we get $\frac{1}{2}\rho(G) \leq |P|$. Note that we do not need the randomization for the proof of the lower bound.

By construction of P, every vertex has degree at most two in P. Hence, all connected components of P are cycles and paths. We claim that at most $\frac{2}{K+2}$ fraction of connected components are cycles in expectation. Since the expected number of connected components is at most $\mathrm{E}\,|P|$, from this we get that the expected number of cycles is at most $2\,\mathrm{E}\,|P|/(K+2)$. By removing one edge from each cycle, we obtain a valid solution for maximum path cover problem. Thus,

$$\mathrm{E}\,|P| - \frac{2\,\mathrm{E}\,|P|}{K+2} = \frac{K}{K+2}\,\mathrm{E}\,|P| \leq \rho(G) \qquad \Rightarrow \qquad \mathrm{E}\,|P| \leq \left(1 + \frac{2}{K}\right) \cdot \rho(G).$$

So it remains to show that at most $\frac{2}{K+2}$ fraction of connected components are cycles in expectation. As we process edges one by one according to the ordering of RGMIS, let A be the set of edges that none of their incident edges are added to the solution of Algorithm 2. By definition of A, if one copy of edge (u, v) is in A, then all other copies of (u, v) are also in A. Therefore, at any point during running RGMIS, if a new component is added to the solution, the edge (u, v) that gets added to the solution occupies (u^0, v^0) with probability at least $\frac{K}{K+2}$ since K copies out of the $K+2$ copies are for (u^0, v^0). Let C_0 be the number of times that the newly added component is

an edge occupying two 0-ports, and C_1 be the number of times that the newly added component is an edge occupying one 0-port and one 1-port. By the above argument, we have

$$\frac{\mathrm{E}[C_0]}{\mathrm{E}[C_0]+\mathrm{E}[C_1]} = \frac{K}{K+2}. \tag{1}$$

Note that after running Algorithm 2, it is possible that the number of connected components is actually smaller than C_0+C_1, since some of the components may merge as the algorithm proceeds. However, by Observation 4.4, two components that their first edge occupies two 0-ports will not merge together. Also, by Observation 4.3, none of the cycle components have an edge that occupies two 0-ports. Therefore, in the end, there exists at most $\mathrm{E}[C_0]+\mathrm{E}[C_1]$ connected components and at least $\mathrm{E}[C_0]$ of them will not be cycles. This completes the proof. □

5 A Local Query Process for Algorithm 2 and its Complexity

In this section, we define a query process to estimate the size of the output of Algorithm 2.

In graph H of Algorithm 2, each vertex corresponds to an edge in the original graph. More precisely, we make $K+2$ copies of each edge (u,v) such that one of the copies corresponds to an edge occupying (u^0,v^1), one for (u^1,v^0), and K for (u^0,v^0). We use $G'=(V,E')$ to show the new graph with these parallel edges. During the course of Algorithm 2, two different edges that share the same endpoint and port cannot appear in the solution together. We use the following definition to formalize this notion.

Definition 5.1 (Conflicting Pair of Edges). Two edges $e,e' \in E'$ that share an endpoint v are conflicting if both e and e' correspond to same port v^i for $i \in \{0,1\}$. We call (e,e') a conflicting pair of edges.

In order to estimate the size of the output of Algorithm 2, we define a vertex oracle that given a vertex v and a permutation π on E', returns the degree of vertex v in the output of Algorithm 2. These are akin to the query processes used before in the works of [2, 30], but are specific to our Algorithm 2.

Algorithm 3: "vertex oracle" $\mathrm{VO}(u,\pi)$ to determine the degree of vertex u in $\mathrm{RGMIS}(G',\pi)$.

Let $e_1=(u,v_1),\ldots,e_r=(u,v_r)$ be the edges incident to u with $\pi(e_1)<\ldots<\pi(e_r)$.
$d \leftarrow 0$
for i in $1\ldots r$ do
    if $\mathrm{EO}(e_i,v_i,\pi)=$ TRUE then $d \leftarrow d+1$;
return d

Algorithm 4: "edge oracle" $\mathrm{EO}(e,u,\pi)$ to determine an edge e is in $\mathrm{RGMIS}(G',\pi)$. Also, u must be an endpoint of e.

if $\mathrm{EO}(e,u,\pi)$ computed before then return the computed result.;
Let $e_1=(u,v_1),\ldots,e_r=(u,v_r)$ be the edges incident to e such that $\pi(e_1)<\ldots<\pi(e_r)<\pi(e)$. Also, (e,e_i) is a conflicting pair for all $1\le i\le r$.
for i in $1\ldots r$ do
    if $\mathrm{EO}(e_i,v_i,\pi)=$ TRUE then return FALSE;
return TRUE

Note that in Line 2 of the Algorithm 4 we only recursively call the function on edges that their label, conflict with edge e since if other edges appear in the RMGIS subgraph, we can still have e in the RGMIS subgraph. Before analyzing the query complexity of the vertex oracle, we prove the correctness of the vertex oracle.

Claim 5.2. For any edge $e = (u, z) \in E'$ that is occupying ports u^i and z^j, if $\text{EO}(e, u, \pi)$ is called while computing $\text{VO}(v, \pi)$, then $\text{EO}(e, u, \pi) = \text{TRUE}$ iff $e \in \text{RGMIS}(G', \pi)$.

Proof. We prove the claim using induction on ranking of edge e. Assume that the claim is true for all edges with ranking smaller than $\pi(e)$. If $\text{EO}(e, u, \pi)$ is called by $\text{EO}(e' = (w, z), z, \pi)$ or directly by $\text{VO}(v, \pi)$, then by definition of Algorithm 4 and Algorithm 3, all edges $e'' = (w', z)$ with $\pi(e'') < \pi(e')$ that are occupying z^j are queried before e' which means that none of them return TRUE. Hence, by induction hypothesis, none of the edges incident to z that are occupying z^j with lower rank are in the $\text{RGMIS}(G', \pi)$. Moreover, $\text{EO}(e, u, \pi)$ calls all incident edges to u with lower rank that are occupying u^i and return TRUE if none of them are in the $\text{RGMIS}(G', \pi)$ by induction hypothesis. Therefore, $\text{EO}(e, u, \pi) = \text{TRUE}$ iff $e \in \text{RGMIS}(G', \pi)$. □

Claim 5.3. Let $v \in V$ and d be the output of $\text{VO}(v, \pi)$. Then d is equal to the degree of vertex v in the subgraph outputted by $\text{RGMIS}(G', \pi)$.

Proof. The observation follows by combining the fact that the vertex oracle queries edges in increasing order and Claim 5.2. □

Let $T(v, \pi)$ denote the number of recursive calls to the edge oracle during the execution of $\text{VO}(v, \pi)$.

Theorem 5.4. For a randomly chosen vertex v and permutation π on E', we have that

$$\text{E}_{v,\pi}[T(v, \pi)] = O(\bar{d} \cdot \log^2 n)$$

where $\bar{d}$ is the average degree of the graph G.

Let $Q(e, v, \pi)$ be the number of $\text{EO}(e, \cdot, \pi)$ calls during the execution of $\text{VO}(v, \pi)$. Moreover, let $Q(e, \pi)$ be the number of $\text{EO}(e, \cdot, \pi)$ calls starting from any vertex. In other words, we have that $Q(e, \pi) = \sum_{v \in V} Q(e, v, \pi)$.

Observation 5.5. For every edge e and permutation π, $Q(e, \pi) \leq O(n^2)$.

Proof. Let $e = \{x, y\}$. For a fixed vertex u, either the vertex oracle $\text{VO}(u, \pi)$ queries the edge oracle for e directly, or through some incident edge e'. Hence, the edge oracle of e is called through at most $(K + 2)(\deg(x) - 1) + (K + 2)(\deg(y) - 1)$ of its incident edges ($K + 2$ appears since each edge has $K + 2$ copies), which implies that $Q(e, u, \pi) \leq (2K + 4)(n - 1) + 1$. Therefore,

$$Q(e, \pi) \leq \sum_{u \in V} Q(e, u, \pi) \leq n\left((2K + 4)(n - 1) + 1\right) \leq O(n^2)$$ □

The main contribution of this section is to show that the expected number of $\text{EO}(e, \pi)$ calls over all permutations π is $O(\log^2 n)$, which is formalized in the following lemma.

Lemma 5.6. For any edge $e \in E'$, we have $\text{E}_\pi[Q(e, \cdot, \pi)] = O(\log^2 n)$.

Assuming the correctness of Lemma 5.6, we can complete the proof of Theorem 5.4.

Proof of Theorem 5.4.

$$\begin{aligned}
\mathrm{E}_{v,\pi}[T(v,\pi)] = \frac{1}{n}\,\mathrm{E}_{\pi}\Big[\sum_{v\in V} T(v,\pi)\Big] &= \frac{1}{n}\,\mathrm{E}_{\pi}\Big[\sum_{v\in V}\sum_{e\in E'} Q(e,v,\pi)\Big] \\
&= \frac{1}{n}\,\mathrm{E}_{\pi}\Big[\sum_{e\in E'}\sum_{v\in V} Q(e,v,\pi)\Big] = \frac{1}{n}\,\mathrm{E}_{\pi}\Big[\sum_{e\in E'} Q(e,\pi)\Big] \\
&= \frac{1}{n}\sum_{e\in E'} \mathrm{E}_{\pi}[Q(e,\pi)] = \frac{1}{n}\sum_{e\in E'} O(\log^2 n) \\
&= \frac{1}{n} O(|E'|\cdot \log^2 n) = O(\bar{d}\cdot \log^2 n).
\end{aligned}$$

□

During the recursive calls to the edge oracle that starts from vertex v, the edges in the stack of recursive calls create a trail.

Observation 5.7. Let $S = (e_1 = (v,u), e_2, \ldots, e_r)$ be the stack of recursive calls starting from vertex v. Then $(e_1, e_2, \ldots, e_r)$ is a trail in G'.

Proof. Since in Line 2 of Algorithm 4, edge oracle only queries incident edges, $(e_1, e_2, \ldots, e_r)$ is a walk. It remains to show that all edges are distinct. Suppose that $e_i = e_j$ for some $i < j$ which implies $\pi(e_i) = \pi(e_j)$. Since the edge oracle queries edges in decreasing order, we have $\pi(e_j) < \pi(e_i)$ which is a contradiction. □

We direct the edges of the trail from v to the other endpoint. We call a trail that starts from v on the graph with edge permutation π, a (v,π)-query-trail. For an edge $e = (x,y)$, let $\vec{e}$ denote the directed edge from x to y and $\overleftarrow{e}$ denote a directed edge from y to x.

Observation 5.8. Let $\vec{P} = (\vec{e_1}, \vec{e_2}, \ldots, \vec{e_k})$ be a (v,π)-query-trail; then $\pi(e_1) > \pi(e_2) > \ldots > \pi(e_k)$.

Proof. During the answering whether an edge is in $\mathrm{RGMIS}(G',\pi)$, Algorithm 4 recursively calls on edges with π values lower than the value of the current edge. Therefore, the stack of recursive calls will be decreasing with respect to π values. □

Let $Q(\vec{e},\pi) \subseteq Q(e,\pi)$ be the set of all query trails that end at $\vec{e}$ (with the same direction). In what follows, we obtain a bound for the query complexity for $\vec{e}$. We use this lemma to prove Lemma 5.6.

Lemma 5.9. For any edge e, we have $\mathrm{E}_{\pi}[Q(\vec{e},\pi)] = O(\log^2 n)$.

Proof of Lemma 5.6. Since $Q(e,\pi) = Q(\vec{e},\pi) \cup Q(\overleftarrow{e},\pi)$, by Lemma 5.9 we have

$$\mathrm{E}_{\pi}[Q(e,\pi)] \leq \mathrm{E}_{\pi}[Q(\vec{e},\pi)] + \mathrm{E}_{\pi}[Q(\overleftarrow{e},\pi)] = O(\log^2 n) + O(\log^2 n) = O(\log^2 n)$$

□

Given a permutation π and a trail $\vec{P} = (\vec{e_1}, \vec{e_2}, \ldots, \vec{e_k})$, we define $\phi(\pi, \vec{P})$ to be another permutation σ over the edges such that:

$$\begin{aligned}
(\sigma(e_1), \sigma(e_2), \ldots, \sigma(e_{k-1}), \sigma(e_k)) &\coloneqq (\pi(e_2), \pi(e_3), \ldots, \pi(e_k), \pi(e_1)) \\
\pi(e') = \sigma(e') \qquad \forall e' \notin \vec{P}
\end{aligned}$$

Given an edge $\vec{e}$, by using the above mapping function we can construct a bipartite graph H with two parts A and B such that each part has $|E'|!$ vertices showing different permutations of

edges. For a permutation $\pi \in A$ and a (v, π)-query-trail $\vec{P}$ that ends at $\vec{e}$ for some arbitrary vertex v, we connect π in A to $\phi(\pi, \vec{P})$ in B. Note that by construction of H, $\deg(\pi_A) = Q(\vec{e}, \pi_A)$ for all $\pi_A \in A$, since we have a unique edge for each query-trail that ends at $\vec{e}$ with permutation π_A. Hence, in order to prove Lemma 5.6, it is sufficient to prove that $\mathrm{E}_{\pi_A \sim A}[\deg_H(\pi_A)] = O(\log^2 n)$. Let $\mathcal{Q}(\vec{e}, \pi)$ be the set of all query-trails for permutation π that ends at $\vec{e}$. Let $\beta = c \log^2 n$ for some large c. We partition permutations into two sets of likely and unlikely permutations called L and U as follows:

$$L \coloneqq \left\{ \pi \in \Pi \middle| \max_{\vec{P} \in \mathcal{Q}(\vec{e}, \pi)} |\vec{P}| \leq \beta \right\} \qquad U \coloneqq \Pi \setminus L.$$

Likely permutations are those permutations that the longest query-trail ending at $\vec{e}$ has length at most β and unlikely permutations are the remaining permutations. Let A_L be the set of vertices corresponding to the likely permutations in A and A_U be the set of vertices corresponding to the unlikely permutations. The intuition behind this partitioning is that the set of unlikely permutations makes up a tiny fraction of all permutations which is formalized in Lemma 5.10.

Lemma 5.10. If c is a large enough constant, then we have $|A_U| \leq |E'|!/n^2$.

Before proving Lemma 5.10, we introduce the parallel implementation of the greedy maximal independent set.

Parallel Randomized Greedy Maximal Independent Set: Let G be a graph and π be a permutation over its edges. In each iteration, we pick all vertices whose rank is less than all their neighbors and remove all their neighbors. We denote the number of rounds in this algorithm until G becomes empty as round complexity and we show it with $\rho(G, \pi)$.

It is clear that the output of the parallel randomized greedy MIS is the same as $\mathrm{RGMIS}(G, \pi)$. We have the following known result about the round complexity of parallel randomized greedy MIS.

Lemma 5.11 ([10, Theorem 3.5]). For a uniformly random chosen permutation π over edges of G, we have $\rho(G, \pi) = O(\log^2 n)$, with probability of at least $1 - \frac{1}{n^2}$.

In order to use the above lemma, we need to show that for an unlikely permutation, the round complexity is large and therefore, small fraction of permutations are unlikely as a result of Lemma 5.11.

Claim 5.12. Let $\vec{P}$ be query-trail in G' with permutation π. Then $\rho(G', \pi) \geq \lfloor \frac{|\vec{P}|}{2} \rfloor$.

Proof. Let $\vec{P} = (\vec{e_1}, \vec{e_2}, \ldots, \vec{e_k})$ be a query-trail. By Observation 5.8, we have $\pi(e_1) > \pi(e_2) > \ldots > \pi(e_k)$, where e_k is the last edge on the trail. Let $\rho(e)$ show the round in which edge e is deleted by the parallel algorithm. If we can show that for $i < k-1$, $\rho(e_i) > \rho(e_{i+2})$, then we have that $\rho(e_2) \geq \lfloor \frac{k}{2} \rfloor$ which completes the proof. We prove it using a contradiction. Assume that $\rho(e_i) \leq \rho(e_{i+2})$ for some $1 < i < k-1$. Note that $\rho(e_{i+1}) \geq \rho(e_i)$, otherwise, when e_{i+1} is deleted from the graph, one of its corresponding ports that is shared with e_i and e_{i+2} was occupied which implies that at least one of e_i and e_{i+2} should be deleted at the same time. Hence, in round $\rho(e_i)$, edge e_{i+1} is still present in the graph. Therefore, e_i is not a local minimum in round $\rho(e_i)$ and is deleted due to presence of an edge e' in the solution. Note that $e' \neq e_{i+1}$ since e_{i+1} is not the minimum edge because e_{i+2} is still in the graph. If e' is only incident to e_i, $\mathrm{EO}(e_{i-1}, \cdot, \pi)$ should call $\mathrm{EO}(e', \cdot, \pi)$ before $\mathrm{EO}(e_i, \cdot, \pi)$ since e' is the local minimum in round $\rho(e_i)$ and therefore $\pi(e') < \pi(e_i)$. If e' is

incident to both e_i and e_{i+1}, $\mathrm{EO}(e_i, \cdot, \pi)$ should call $\mathrm{EO}(e', \cdot, \pi)$ before $\mathrm{EO}(e_{i+1}, \cdot, \pi)$ since e' is local minimum at round $\rho(e_i)$ and therefore $\pi(e') < \pi(e_{i+1})$. In both cases, the edge oracle terminates and will not query edge e_{i+2}. Hence, the assumption that $\rho(e_i) \leq \rho(e_{i+2})$ leads to a contradiction and the proof is complete. □

Now we are ready to prove Lemma 5.10.

Proof of Lemma 5.10. For each unlikely permutation $\pi \in U$, there exists a query-trail of length larger than β. By Claim 5.12, we have $\rho(G, \pi) \geq \lfloor \frac{\beta+1}{2} \rfloor$. Since $\beta = c \log^2 n$, by choosing c large enough and Lemma 5.11, we have that $|U|/|\Pi| \leq 1/n^2$. Therefore, $|U| \leq |E'|!/n^2$ which implies that $|A_U| \leq |E'|!/n^2$ since A_U represents vertices that correspond to unlikely permutations. □

Next, we show that each vertex $\pi_B \in B$, has at most β neighbors between likely permutations in part A in bipartite graph H.

Lemma 5.13. Let π_Y be a vertex in Y. Then π_Y has most β neighbors in X_L.

Before proving this lemma, we show how we can prove Lemma 5.9 using Lemma 5.10 and Lemma 5.13.

Proof of Lemma 5.9. Note that by Observation 5.5, degree of each vertex $\pi_A \in A$ is at most $O(n^2)$. Combining Lemma 5.10, we have

$$E(A_U, B) \leq |E'|!/n^2 \cdot O(n^2) \leq O\big(|E'|!\big).$$

Moreover, by Lemma 5.13, each vertex $\pi_B \in B$ has at most $O(\beta)$ neighbors in A_L. Since H is a bipartite graph, $E(A_L, B) \leq O(\beta) \cdot |A_L|$. Therefore, sum of degrees of all vertices in A is at most

$$E(A_L, B) + E(A_U, B) \leq O(\beta) \cdot |A_L| + O(|E'|!) \leq O(\beta \cdot |E'|!).$$

For a random vertex in A, the expected degree is $\frac{O(\beta \cdot |E'|!)}{|E'|!} = O(|E'|)$. Combining with $\beta = c \log^2 n$ and $\deg(\pi_A) = Q(\vec{e}, \pi_A)$ completes the proof. □

The rest of this section, we prove Lemma 5.13. Before proving Lemma 5.13, we prove that if two different query-trails that are mapped to two different permutations of A_L to $\pi_B \in B$ by ϕ, the shorter query-trail must be subgraph of the longer one.

Lemma 5.14. Let π and π' be two different permutations, and $\vec{P}$ and $\vec{P}'$ be (v, π)- and (v', π')-query-trail, respectively, that both end at edge $\vec{e}$. If $\phi(\pi, \vec{P}) = \phi(\pi', \vec{P}')$ and $|\vec{P}| \geq |\vec{P}'|$, then $\vec{P}'$ is a subgraph of $\vec{P}$.

We prove this lemma by series of observations and claims. Let $\vec{P} = (\vec{e_k}, \ldots, \vec{e_1})$ and $\vec{P}' = (\vec{e_r}', \ldots, \vec{e_1}')$ such that $e = e_1 = e'_1$. If $\vec{P}'$ is not a subgraph of $\vec{P}$, then it must branch after an edge $\vec{e_b}'$. This means that $\vec{e_i} = \vec{e_i}'$ for $i \leq b$ and $\vec{e_{b+1}} \neq \vec{e_{b+1}}'$. Note that $\vec{e_{b+1}}$ and $\vec{e_{b+1}}'$ can be copy of the same edge.

Observation 5.15. Let π be a random permutation over E'. For a (u, π)-query-trail, if f and f' are two consecutive edges in the trail, then (f, f') is a conflicting pair.

Proof. Since the edge oracle calls $\text{EO}(f', \cdot, \pi)$ in $\text{EO}(f, \cdot, \pi)$, (f, f') must be a conflicting pair. □

Observation 5.16. Let f_1, f_2, f_3 be three different edges incident to some vertex u and let π be a random permutation over E'. Let $\vec{P_1}$ be a (x, π)-query-trail that calls $\text{EO}(f_3, \cdot, \pi)$ in $\text{EO}(f_1, \cdot, \pi)$. Also, let $\vec{P_2}$ be a (y, π')-query-trail that calls $\text{EO}(f_3, \cdot, \pi')$ in $\text{EO}(f_2, \cdot, \pi')$. Then (f_1, f_2) is a conflicting pair.

Proof. By Observation 5.15, (f_1, f_3) is a conflicting pair. Assume that both f_1 and f_3 occupied port u^i. Moreover, since (f_2, f_3) is a conflicting pair, then f_2 is also occupying u^i. Therefore, (f_1, f_2) is a conflicting pair. □

Observation 5.17. $\pi(e_b) = \pi'(e_{b+1})$.

Proof. Since $\vec{e_{b+1}}$ is not in $\vec{P'}$, we have that $\phi(\pi', \vec{P'})(e_{b+1}) = \pi'(e_{b+1})$. Also, $\phi(\pi, \vec{P})(e_{b+1}) = \pi(e_b)$ since $\phi(\pi, \vec{P})$ shifts edges of the trail $\vec{P}$ by one. Given that permutation $\phi(\pi, \vec{P})$ is equal to $\phi(\pi', \vec{P'})$, we have $\pi(e_b) = \pi'(e_{b+1})$. □

Without loss of generality, we can assume that $\pi(e_b) \leq \pi'(e_b)$ since we did not make any difference between π and π' until this point.

Observation 5.18. $\pi'(e_{b+1}) < \pi'(e_b)$.

Proof. By combining Observation 5.17, our assumption that $\pi(e_b) \leq \pi'(e_b)$, and the fact that π' is a permutation, we have that $\pi'(e_{b+1}) < \pi'(e_b)$. □

Claim 5.19. If $\pi(f) < \pi(e_b)$ or $\pi'(f) < \pi(e_b)$ for some edge $\vec{f}$, then $\pi(f) = \pi'(f)$.

Proof. There are five different possible cases for f:

- $\vec{f} \notin \vec{P} \cup \vec{P'}$: Since ϕ only changes the edge on the query-trail and $\phi(\pi, \vec{P}) = \phi(\pi', \vec{P'})$, we have $\pi(f) = \pi'(f)$.
- $\vec{f} \in \{\vec{e_1}, \ldots, \vec{e}_{b-1}\}$: Since $\phi(\pi, \vec{P})(e_{i+1}) = \phi(\pi', \vec{P'})(e_{i+1})$ for $1 \leq i < b$, we have $\pi(e_i) = \pi'(e_i)$. Hence, $\pi(f) = \pi'(f)$.
- $\vec{f} = \vec{e_b}$: In this case, condition $\pi(f) < \pi(e_b)$ does not hold since $\pi(f) = \pi(e_b)$. Also, $\pi'(f) = \pi'(e_b) \geq \pi(e_b)$. Therefore, condition $\pi'(f) < \pi(e_b)$ does not hold.
- $\vec{f} \in \{\vec{e}_{b+1}, \ldots, \vec{e_k}\}$: By Observation 5.8, we have that $\pi(f) > \pi(e_b)$. Therefore, condition $\pi(f) < \pi(e_b)$ does not hold. Let $\vec{f} = \vec{e_i}$ for $i > b$. Since $\phi(\pi, \vec{P}) = \phi(\pi', \vec{P'})$, we have $\pi'(f) = \pi(e_{i-1}) \geq \pi(e_b)$. Therefore, none of the conditions in the claim statement holds.
- $\vec{f} \in \{\vec{e_{b+1}}', \ldots, \vec{e_r}'\}$: By Observation 5.8, we have that $\pi'(f) > \pi'(e_b)$. Combining by our assumption that $\pi'(e_b) \geq \pi(e_b)$, we have $\pi'(f) \geq \pi(e_b)$. Let $\vec{f} = \vec{e_i}'$ for $i > b$. Since $\phi(\pi, \vec{P}) = \phi(\pi', \vec{P'})$, we have that $\pi(f) = \pi'(e'_{i-1}) \geq \pi'(e_b) \geq \pi(e_b)$. Therefore, none of the conditions in the claim statement holds.

The proof is thus complete. □

Claim 5.20. $e_{b+1} \in \text{RGMIS}(G', \pi')$.

Proof. We prove the claim by contradiction. Assume that $e_{b+1} \notin \text{RGMIS}(G', \pi')$. Hence, there exists an edge f which is incident to e_{b+1} such that $\pi'(f) < \pi'(e_{b+1})$. Thus, $\text{EO}(e_{b+1}, \cdot, \pi')$ will recursively call $\text{EO}(f, \cdot, \pi')$. Let f be incident to e_i and e_{i+1} for $i \in \{b, b+1\}$. In the query-trail $\vec{P}$, $\text{EO}(e_{i+1}, \cdot, \pi)$ calls $\text{EO}(e_i, \cdot, \pi)$. Therefore, using the Observation 5.16, we have that (f, e_i) is a conflicting pair. Note that by Observation 5.17, we have $\pi'(f) < \pi(e_b)$. Hence, $\pi(f) = \pi'(f) < \pi(e_b)$ by Claim 5.19. Since both permutations are identical for ranks lower than $\pi(e_b)$, edge f must appear in $\text{RGMIS}(G', \pi)$ and the query-trail $\vec{P}$ is not a valid query-trail since $\text{EO}(e_i, \cdot, \pi)$ terminates upon calling $\text{EO}(f, \cdot, \pi)$ (see Figure 2). □

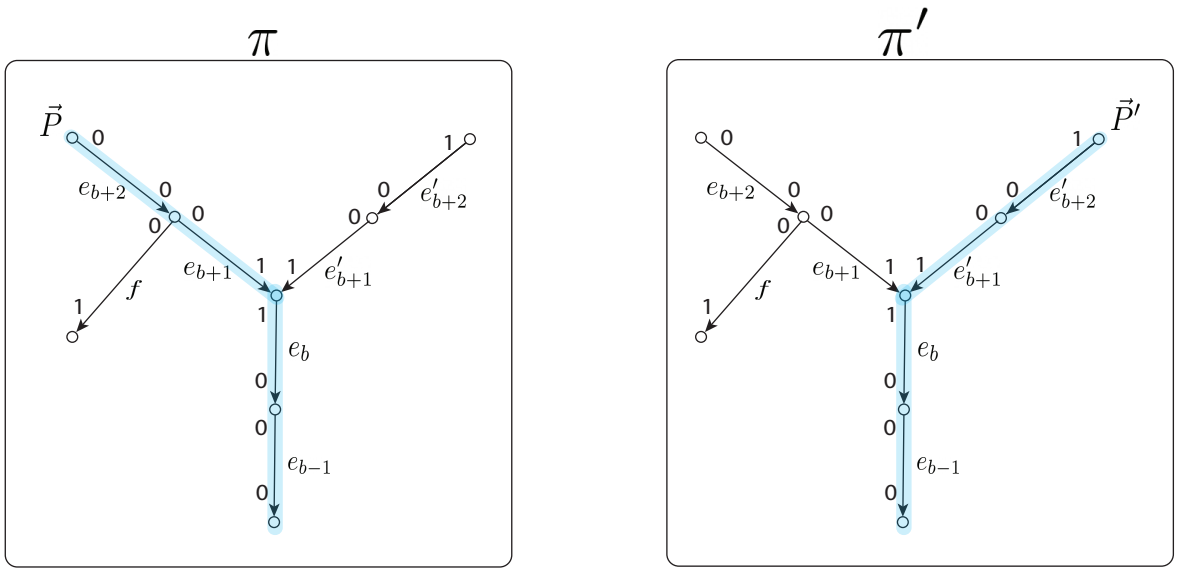


Figure 2: Illustration of proof of Claim 5.20. The highlighted blue trails show query-trails $\vec{P}$ and $\vec{P}'$. Query-trail $\vec{P}$ is not valid since $\text{EO}(e_i, \cdot, \pi)$ terminates upon calling $\text{EO}(f, \cdot, \pi)$.

Proof of Lemma 5.14. We prove that query-trail $\vec{P}'$ is not a valid (v, π')-query-trail. Note that by Observation 5.18, $\text{EO}(e'_{b+1}, \cdot, \pi')$ calls $\text{EO}(e_{b+1}, \cdot, \pi')$ before $EO(e_b, \cdot, \pi')$. Thus, by Claim 5.20, $\text{EO}(e_{b+1}, \cdot, \pi')$ will return TRUE and execution of $\text{EO}(e'_{b+1}, \cdot, \pi')$ terminates at this point. Therefore, query-trail $\vec{P}'$ is a subgraph of query-trail $\vec{P}$. □

Now we are ready to complete the proof of Lemma 5.13.

Proof of Lemma 5.13. For each edge between $\pi_A \in A_L$ and $\pi_B \in B$ in graph H, we write a label $\chi(\pi_A, \pi_B)$ on the edge which is equal to the length of the query-trail corresponding to this edge in H. By Lemma 5.14, all the labels for edges of a fixed vertex $\pi_B \in B$ that are incident to A_L should be different. Moreover, by the definition of likely permutations, all query-trails of permutation A_L have length less than or equal to β. Thus, each vertex $\pi_B \in B$ has at most β neighbors in A_L. □

6 Our Estimator for Maximum Path Cover

In this section, we use the oracle of the previous section to estimate the number of edges in the output of Algorithm 2. In Section 5, we provide a lower bound on the number of recursive calls to our local query process. Note that this bound does not necessarily imply the same running time algorithm. For example, if we generate the whole permutation over all copies of edges before running the algorithm, it takes $\Theta(m)$ which is no longer sublinear. Using by now standard ideas of the literature, we show in Section A how we can implement the query process in almost the same running time (multiplied by a polylogarithmic factor) which is formalized in the following lemma.

Lemma 6.1. There exists a data structure that given a graph G in the adjacency list format, (implicitly) fixes a random permutation π over its edges. Then for any vertex v, the data structure returns the degree of vertex v in the subgraph P produced by Algorithm 2 according to a random permutation π. Each query v to the data structure is answered in $\tilde{O}(T(v,\pi))$ time w.h.p. where $T(v,\pi)$ is as defined in Section 5.

Note that in our local query process, we need access to the adjacency list of weight-one edges. So the challenge that arises here is how to estimate the size of the output of Algorithm 2 in the adjacency matrix model. We present a reduction from adjacency matrix to adjacency list that appeared in the literature [2]. In this reduction, each query to the adjacency list can be implemented with $O(1)$ queries to the adjacency matrix and still we are able to estimate the maximum path cover with some additive error.

Let $\gamma = 16Kn$. We construct a graph $\hat{G} = (V_{\hat{G}}, E_{\hat{G}})$ for weight-one edges of graph G as follows:

- $V_{\hat{G}}$ is the union of V_1, V_2 and $U_1, U_2, \ldots, U_n$ such that:
  - V_1 and V_2 are two copies of the vertex set of the original graph G.
  - U_i is a vertex set of size γ for each $i \in [n]$.
- We define the edge set such that degree of each vertex is in $\{1, n, n+\gamma\}$:
  - Degree of each vertex $v \in V_1$ is n. The i-th neighbor of v is the i-th vertex in V_1 if $(v,i) \in E$, otherwise its i-th neighbor is the i-th vertex in V_2 for $i \leq n$. Note that graph $(V_1, E_H \cap (V_1 \times V_1))$ is isomorphic to G.
  - Degree of each vertex $v \in V_2$ is $n+\gamma$. The i-th neighbor of v is the i-th vertex in V_2 if $(v,i) \in E$, otherwise, its i-th neighbor is the i-th vertex in V_1 for $i \leq n$. For all $n < i \leq n+\gamma$, the i-th neighbor of v is i-th vertex in U_v.
  - Degree of each vertex $u \in U_i$ is one for $i \in [n]$. The only neighbor of u is the i-th vertex of V_2.

By the construction of $\hat{G}$, the only neighbor of $v \in \bigcup_{i=1}^{n} U_i$ can be determined without any query to the adjacency matrix. Also, the i-th neighbor of each vertex in $V_1 \cup V_2$ can be determined with one query.

Observation 6.2. For each vertex $v \in V_{\hat{G}}$ and $i \in [\deg_{\hat{G}}(v)]$, the i-th neighbor of vertex v can be determined using at most one query to the adjacency matrix.

Fix a vertex $v \in V_2$. When we run Algorithm 2, intuitively with high probability the first edge that is incident to v and occupies port v^0 is between v and $u \in U_v$. Furthermore, with high probability the first two edges that are incident to v and occupies port v^1 are between v and $u \in U_v$. A vertex $v \in V_2$ is an abnormal vertex if the above properties do not hold for v. Let $R \in V_2$ be the set of abnormal vertices. In the following observation, we show that for each vertex $v \in V_2 \setminus R$, all incident edges of v in the output of Algorithm 2 are between v and vertices of U_v.

Claim 6.3. $\mathrm{E}_\pi |R| \leq n/(4K)$

Proof. Fix a vertex $v \in V_2$. For a random permutation over copies of edges of $\hat{G}$, the first incident edge to v that occupies port v^0 is between v and U_v with a probability of at least $\frac{(K+1)\gamma}{(n+\gamma)(K+1)} \geq 1 - \frac{1}{8K}$. Moreover, the first two edges that occupy v^1 are between v and U_v with probability of at least $\frac{\gamma(\gamma-1)}{(n+\gamma)(n+\gamma-1)} \geq 1 - \frac{1}{8K}$. Since both events are independent, the probability of v not being an abnormal vertex is at least

$$\left(1 - \frac{1}{8K}\right)^2 \geq 1 - \frac{1}{4K},$$

which implies $\mathrm{E}_\pi |R| \leq n/(4K)$. □

Claim 6.4. For each $v \in V_2 \setminus R$, all incident edges of v in the output of Algorithm 2 are between v and vertices of U_v.

Proof. By definition of an abnormal vertex, let the first edge in the permutation incident to v be between v and $w \in U_v$ which occupies v^0. Since all copies of edges incident to w are between v and w, this edge will be added to the solution of Algorithm 2. Moreover, we know that the first two edges that are incident to v and occupy port v^1 are between v and U_v. Let these two edges be (v, u_1) and (v, u_2) where $u_1, u_2 \in U_v$. Note that the only way that (v, u_1) is not added to the solution of Algorithm 2 is when $u_1 = w$. In this case, since there is only one copy for each edge that occupied port v^1, then $u_2 \neq w$. Therefore, Algorithm 2 adds (v, u_2) to its output if it has not added (v, u_1). Since both ports of v are occupied in this case, all incident edges of v in the output of Algorithm 2 are between v and vertices of U_v. □

Observation 6.5. Let P be the output of Algorithm 2 on $\hat{G}$. Then

$$\frac{1}{2}\rho(\hat{G}[V_1 \cup R]) \leq \mathrm{E}\,|P \cap (V_1 \cup R) \times (V_1 \cup R)| \leq (1 + \frac{2}{K}) \cdot \rho(\hat{G}[V_1 \cup R]).$$

Proof. By Claim 6.4, if we run Algorithm 2 on $\hat{G}$, for any vertex $v \in V_2 \setminus R$, all incident edges of v in the output are between v and U_v. Hence, none of the edges between $V_2 \setminus R$ and $V_1 \cup R$ will be added to the output of Algorithm 2. Since, the permutation over edges of $V_1 \cup R$ is uniformly at random, by Lemma 4.5, we obtain the claimed bound. □

In the above sequence of observations, we show that there are few abnormal vertices in V_2, which implies that most of the incident edges to vertices of V_1 in the output of Algorithm 2 are in $\hat{G}[V_1]$ (only those between V_1 and R violate this property). Therefore, a natural way to estimate the number of edges in the output of Algorithm 2 on G, is to estimate the number of edges in $\hat{G}[V_1]$ in the output of Algorithm 2 on $\hat{G}$. With this intuition in mind, we need to bound the query complexity of the algorithm for a random vertex in V_1.

Claim 6.6. Let v be a random vertex in V_1 and π be a random permutation over edges of graph that is created by copying $E_{\hat{G}}$ according to Algorithm 2. Then

$$\mathrm{E}_{v \sim V_1, \pi}[T(v, \pi)] = \tilde{O}(n).$$

Proof. By Theorem 5.4, we have that

$$\mathrm{E}_{v\sim V_{\hat{G}},\pi}[T(v,\pi)] = O(\frac{K\cdot|E_{\hat{G}}|}{|V_{\hat{G}}|}\cdot\log^2|V_{\hat{G}}|).$$

Summing over all vertices in $V_{\hat{G}}$, we obtain

$$\sum_{v\in V_{\hat{G}}}\mathrm{E}_{\pi}[T(v,\pi)] = O(K\cdot|E_{\hat{G}}|\cdot\log^2|V_{\hat{G}}|) = \tilde{O}(n^2),$$

since $|V_{\hat{G}}| = O(n^2)$, $K = O(1/\varepsilon)$, and $|E_{\hat{G}}| = O(n^2)$. Therefore, for a random vertex in V_1, we get

$$\mathrm{E}_{v\sim V_1,\pi}[T(v,\pi)] \leq \left(\sum_{v\in V_{\hat{G}}}\mathrm{E}_{\pi}[T(v,\pi)]\right)/|V_1| = \tilde{O}(n)$$

Algorithm 5: Final algorithm for maximum path cover.

1 Let $\hat{G} = (V_{\hat{G}}, E_{\hat{G}})$ as described above.
2 $r \leftarrow 192\cdot K^2\cdot\log n$.
3 Sample r vertices $u_1, u_2, \ldots, u_r$ uniformly at random from V_1 with replacement.
4 Sample r ports $p_1, p_2, \ldots, p_r$ uniformly at random from $\{0,1\}$.
5 Run vertex oracle for each u_i and let X_i be the indicator if port $u_i^{p_i}$ is occupied with an edge in $\hat{G}[V_1]$ in output of Algorithm 2.
6 Let $X = \sum_{i\in[r]} X_i$ and $f = X/r$.
7 Let $\tilde{\rho} = \frac{K}{2(K+2)}\cdot(f\cdot n - \frac{n}{4K})$.
8 return $\tilde{\rho}$

Lemma 6.7. Let $\tilde{\rho}$ be the output of Algorithm 5 on input graph G. With high probability,

$$\left(\frac{1}{2}-\frac{1}{K}\right)\cdot\rho(G) - \frac{n}{K} \leq \tilde{\rho} \leq \rho(G),$$

where K is the parameter which is defined in Algorithm 2.

Proof. Let $\hat{P}$ be the set of edges outputted by Algorithm 2 on $\hat{G}$ with both endpoints in V_1. By Lemma 4.5, we have that $\mathrm{E}\,|\hat{P}| \leq (1+\frac{2}{K})\cdot\rho(G)$. Furthermore, by Claim 6.3 and the fact that the degree of each vertex in the output of Algorithm 2 is at most two, in the output of Algorithm 2 on $\hat{G}[V_1\cup R]$ we have at most $n/(2K)$ edges with one endpoint in R. Hence, combining with Lemma 4.5 and Observation 6.5 we get

$$\frac{1}{2}\rho(G) - \frac{n}{2K} \leq \mathrm{E}\,|\hat{P}| \leq (1+\frac{2}{K})\cdot\rho(G). \tag{2}$$

Since each edge in the output of Algorithm 2 counted twice in Algorithm 5, we have

$$\mathrm{E}[X_i] = \Pr[X_i = 1] = \frac{2\,\mathrm{E}\,|\hat{P}|}{n},$$

and,

$$\mathrm{E}[X] = \frac{2r\,\mathrm{E}\,|\hat{P}|}{n}. \tag{3}$$

Since X is sum of r independent random variables, by Chernoff bound (Proposition 3.2) we get

$$\Pr[|X - \mathrm{E}[X]| \leq \sqrt{6\,\mathrm{E}[X]\log n}] \leq 2\exp\left(-\frac{6\,\mathrm{E}[X]\log n}{3\,\mathrm{E}[X]}\right) = \frac{2}{n^2}.$$

Combining $fn = Xn/r$ and the above bound, with probability of at least $1 - 2/n^2$ we have

$$\begin{aligned} fn \in \frac{n(\mathrm{E}[X] \pm \sqrt{6\,\mathrm{E}[X]\log n})}{r} &= \frac{n\,\mathrm{E}[X]}{r} \pm \sqrt{\frac{6n^2\,\mathrm{E}[X]\log n}{r^2}} \\ &= 2\,\mathrm{E}\,|\hat{P}| \pm \sqrt{\frac{12n\,\mathrm{E}\,|\hat{P}|\log n}{r}} && \text{(By (3))} \\ &= 2\,\mathrm{E}\,|\hat{P}| \pm \sqrt{\frac{n\,\mathrm{E}\,|\hat{P}|}{16K^2}} && (\text{Since } r = 192\cdot K^2\cdot\log n) \\ &\in 2\,\mathrm{E}\,|\hat{P}| \pm \frac{n}{4K} && (\text{Since } \mathrm{E}\,|\hat{P}| \leq n). \end{aligned}$$

Since, $\tilde{\rho} = \frac{K}{2(K+2)} \cdot (f \cdot n - \frac{n}{4K})$, hence

$$\frac{K}{K+2}\left(\mathrm{E}\,|\hat{P}| - \frac{n}{2K}\right) \leq \tilde{\rho} \leq \frac{K}{K+2}\cdot \mathrm{E}\,|\hat{P}|.$$

Combining with (2), implies the claimed bound. □

Theorem 6.8. Given an adjacency matrix access for input graph G, there exists a randomized algorithm that w.h.p. runs in $\widetilde{O}(n)$ time and produces an estimate $\tilde{\rho}$, such that

$$\left(\frac{1}{2} - \varepsilon\right)\cdot\rho(G) - \varepsilon n \leq \tilde{\rho} \leq \rho(G).$$

Proof. Let $K = \frac{1}{\varepsilon}$ and $\tilde{\rho}$ be the output of Algorithm 5 on G. In Algorithm 5, by combining Lemma 6.1 and Claim 6.6, the running time for each sample is $\tilde{O}(n)$. Since the number of samples is $r = 192K^2\log n$, and K is a constant, the total running time of the algorithm is $\tilde{O}(n)$. Moreover, by Lemma 6.7 we get the approximation ratio in the statement. □

7 Our Estimator for (1,2)-TSP

In this section, we use the algorithm for estimating the size of maximum path cover as a black box to estimate the size of (1,2)-TSP. First, note that if there is no Hamiltonian cycle with weight one edges of the graph, then the set of weight-one edges of the graph (1,2)-TSP is a solution for maximum path cover of graph G. Also, in the case that there exists a Hamiltonian cycle, then the size of maximum path cover is $n-1$. Moreover, if P^* is the maximum path cover of a graph G, then it is possible to create a TSP by connecting these paths using edges with weight two. This intuition helps to formalize the following observation.

Observation 7.1. Let $\tau(V)$ be the cost of (1,2)-TSP of graph $G = (V, E)$. Then, we have

$$2n - \rho(G) - 1 \leq \tau(V) \leq 2n - \rho(G).$$

Now we are ready to present the final algorithm for estimating (1,2)-TSP.

Algorithm 6: Final algorithm for (1,2)-TSP.

1 Construct $\hat{G} = (V_{\hat{G}}, E_{\hat{G}})$ implicitly as desribed in Section 6.
2 Let $\tilde{\rho}$ be the output of Algorithm 5 on $\hat{G}$.
3 $\tilde{\tau} = 2n - \tilde{\rho}$
4 return $\tilde{\tau}$

Lemma 7.2. Let $\tilde{\tau}$ be the output of Algorithm 6 and $\tau(V)$ be the cost of (1,2)-TSP of graph $G = (V, E)$. With high probability,

$$\tau(V) \leq \tilde{\tau} \leq (\frac{3}{2} + \frac{4}{K}) \cdot \tau(V),$$

where K is the parameter which is defined in Algorithm 2.

Proof. By Observation 7.1, we have $2n - \rho(G) - 1 \leq \tau(V) \leq 2n - \rho(G)$. Algorithm 6 outputs $\tilde{\tau} = 2n - \tilde{\rho}$ as the estimate, where $\tilde{\rho}$ is the output of Algorithm 5. Hence, by Lemma 6.7, we have $2n - \tilde{\rho} \geq 2n - \rho(G)$. Also, by Lemma 6.7, we have

$$\begin{aligned}
2n - \tilde{\rho} &\leq 2n - (\frac{1}{2} - \frac{1}{K}) \cdot \rho(G) + \frac{n}{K} \\
&\leq 3n - \frac{3\rho(G)}{2} + \frac{4n}{K} - \frac{2\rho(G)}{K} - 1 && \text{(Since } \rho(G) < n\text{)} \\
&\leq (\frac{3}{2} + \frac{4}{K})(2n - \rho(G) - 1) && \text{(Since } K \ll n\text{)} \\
&\leq (\frac{3}{2} + \frac{4}{K}) \cdot \tau(V) && \text{(Since } \tau(V) = 2n - \rho(G)\text{)},
\end{aligned}$$

which completes the proof. □

Theorem 7.3. Let $\tau(V)$ be the cost of (1,2)-TSP of graph $G = (V, E)$. For any $\varepsilon > 0$, there exists an algorithm that estimate the cost of (1,2)-TSP, $\tilde{\tau}$, such that

$$\tau(V) \leq \tilde{\tau} \leq (\frac{3}{2} + \varepsilon) \cdot \tau(V),$$

w.h.p in $\tilde{O}(n)$ running time.

Proof. We choose $K = \frac{4}{\varepsilon}$. By Lemma 7.2, if $\tilde{\tau}$ is the output of Algorithm 6, we get

$$\tau(V) \leq \tilde{\tau} \leq (\frac{3}{2} + \varepsilon) \cdot \tau(V).$$

Also, since the running time of Algorithm 6 is the same as the running time of Algorithm 5, by Theorem 6.8, the total running time is $\tilde{O}(n)$, which completes the proof. □

8 Our Estimator for Graphic TSP

In this section, we use our algorithm for estimating the size of maximum path cover to estimate the size of graphic TSP. In a recent work, Chen et al. [11] showed that it is possible to obtain a (27/14)-approximate algorithm for graphic TSP by estimating the matching size and the number of biconnected components in the graph. Since the size of graphic TSP is at most $2n$ (the cost of MST is $n-1$), they proved that if a graph has large matching and a few biconnected components, the cost of graphic TSP is significantly lower than $2n$. Since estimating the number of biconnected components is not an easy task in sublinear time, they use a proxy quantity that can be estimated in sublinear time.

We show that if we use our estimator for maximum path cover as a black-box instead of matching estimator in algorithm of [11], we can improve the approximation ratio to 19/10. Moreover, we show that we can estimate the number of bridges in $\tilde{O}(n)$. We exploit this estimation for further improvement to get a 11/6-approximate algorithm for graphic TSP.

Chen et al. [11] introduced the following definition of bad vertex as a proxy for estimating the number of biconnected components.

Definition 8.1 (Bad Vertex). We say a vertex $v \in V$ is a bad vertex, if one of the following holds:

- degree of v is 1,
- v is a cut vertex with degree 2.

In the following series of lemmas, we bound the cost of graphic TSP based on the size of maximum path cover and number of bad vertices. Almost all the steps of this part are similar to the algorithm for graphic TSP of [11] — except the path cover subroutine that we use instead of maximal matching subroutine. We restate some of the useful lemmas to achieve the approximation bound that the black-box algorithm can get, and in the next subsection we improve this bound. First, we prove that if the size of the maximum path cover is small, the cost of graphic TSP is bounded away from n.

Claim 8.2. If the size of maximum path cover of graph G is at most ρ, then the cost of graphic TSP is at least $2n - \rho$.

Proof. Let $(v_0, v_2, \ldots, v_n = v_0)$ be the optimal graphic TSP of graph G. Note that the subgraph induced by weight-one edges of this cycle is a solution for path cover. Hence, at most ρ edges in cycle $(v_0, v_2, \ldots, v_n = v_0)$ have weight one. All the remaining edges have a weight of at least two which implies the claimed bound. □

Furthermore, the following lemma from [11], provides a lower bound for a graphic TSP of graph in terms of number of bad vertices.

Lemma 8.3 ([11, Lemma 2.8]). If the number of bad vertices of graph G is at least β, then the cost of graphic TSP is at least $n + \beta - 2$.

Chen et al. [11] showed that in a biconnected graph, if there exists a matching of large size, the cost of graphic TSP is significantly smaller than $2n$.

Lemma 8.4 ([11, Lemma 2.7]). Let G be a graph and M be a matching of G. Then the cost of graphic TSP is at most $2n - |M|$.

Lemma 8.5 ([11, Lemma 2.11]). Let G be a graph and M' be a matching that none of its edges is a bridge. Then the cost of graphic TSP is at most $2n - \frac{2}{3}|M'|$.

We now upper bound the cost of graphic TSP in terms of size of maximum path cover and number of bad vertices.

Lemma 8.6. If the size of maximum path cover of graph G is $\rho(G)$ and it has β bad vertices, then the cost of graphic TSP is at most $2n - \frac{1}{5}(\rho(G) - 2\beta)$.

Proof. Let l be the number of non-trivial biconnected components and M' be a maximum matching in graph G that none of its edges is a bridge. Also, let B be the number of bridges in G. By the proof of Lemma 2.9 of [11], the cost of graphic TSP is at most $\min\{2n - \frac{2}{3}|M'|, 2n - l\}$. Note that there are at least $\rho(G) - B$ edges of the maximum path cover that are not bridge. Since all non-bridge edges of the maximum path cover are still union of several disjoint paths, there exists a matching with size of at least half of the edges of these paths. Hence, there exist a matching of size at least $\frac{1}{2}(\rho(G) - B)$. On the other hand, in the proof of the same lemma, they showed that $l \geq \frac{B}{2} - \beta$ which implies that the cost of graphic TSP is at most

$$\min\left\{2n - \frac{2}{3}|M'|, 2n - l\right\} \leq \min\left\{2n - \frac{1}{3}(\rho(G) - B), 2n - \frac{B}{2} + \beta\right\}.$$

There are two possible cases:

- If $B \leq \frac{2}{5}\rho(G) + \frac{6}{5}\beta$, then we have
$$2n - \frac{1}{3}(\rho(G) - B) \leq 2n - \frac{1}{3}(\rho(G) - \frac{2}{5}\rho(G) - \frac{6}{5}\beta) = 2n - \frac{1}{5}(\rho(G) - 2\beta).$$
- If $B > \frac{2}{5}\rho(G) + \frac{6}{5}\beta$, then we have
$$2n - \frac{B}{2} + \beta \leq 2n - \frac{1}{5}\rho(G) - \frac{3}{5}\beta + \beta = 2n - \frac{1}{5}(\rho(G) - 2\beta).$$

Therefore, the cost of graphic TSP is at most

$$\min\left\{2n - \frac{1}{3}(\rho(G) - B), 2n - \frac{B}{2} + \beta\right\} \leq 2n - \frac{1}{5}(\rho(G) - 2\beta). \quad \square$$

Now we are ready to introduce the first algorithm for estimating the cost of graphic TSP, which uses our maximum path cover subroutine instead of the matching subroutine as a black-box. In Algorithm 7, we first estimate the size of the maximum path cover and the number of bad vertices of the graph and report the graphic TSP cost in terms of the two estimations. The subroutine used for counting number of bad vertices is similar to the one in section 2.2 of [11].

Lemma 8.7 ([11]). Let β be the number of bad vertices. For any constant $\varepsilon > 0$, there exists an algorithm that w.h.p estimates the number of bad vertices $\tilde{\beta}$, such that $\beta \leq \tilde{\beta} \leq \beta + \varepsilon n$, in $\tilde{O}(n)$ running time.

Lemma 8.8. Let $\tilde{\tau}$ be the output of Algorithm 7 and $\tau(V)$ be the cost of graphic TSP of graph $G = (V, E)$. With high probability,

$$\tau(V) \leq \tilde{\tau} \leq (\frac{19}{10} + \frac{1}{K}) \cdot \tau(V),$$

where K is the parameter which is defined in Algorithm 2.

Algorithm 7: First algorithm for graphic TSP.

1 Construct $\hat{G} = (V_{\hat{G}}, E_{\hat{G}})$ implicitly as desribed in Section 6.
2 Let $\tilde{\rho}$ be the output of Algorithm 5 on $\hat{G}$.
3 Let $\tilde{\beta}$ be the estimate of number of bad vertices.
4 $\tilde{\tau} = 2n - \frac{1}{5}(\tilde{\rho} - 2\tilde{\beta})$
5 return $\tilde{\tau}$

Proof. Let β be the number of bad vertices. By Lemma 6.7 and Lemma 8.7 $\tilde{\rho} \leq \rho(G)$ and $\beta \leq \tilde{\beta}$. Hence, we have $\tau(V) \leq \tilde{\tau}$ by Lemma 8.6.

By Lemma 6.7 and Lemma 8.7, we can estimate $\rho(G)$ and β such that $\left(\frac{1}{2} - \frac{1}{K}\right)\rho(G) - \frac{n}{K} \leq \tilde{\rho}$ and $\tilde{\beta} \leq \beta + \frac{n}{K}$, if we choose $\varepsilon = \frac{1}{K}$. Thus, we have

$$\begin{aligned}\tilde{\tau} &\leq 2n - \frac{1}{5}\left((\frac{1}{2} - \frac{1}{K}) \cdot \rho(G) - \frac{n}{K} - 2(\beta + \frac{n}{K})\right) \\ &\leq 2n - \frac{1}{5}(\frac{\rho(G)}{2} - 2\beta) + \frac{4n}{5K}. && \text{(Since } \rho(G) \leq n\text{).}\end{aligned}$$

On the other hand, assume that the approximation ratio that the algorithm obtains is $\alpha + 1/K$ for some $\alpha \leq 2$. Thus, we get

$$\begin{aligned}(\alpha + \frac{1}{K}) \cdot \tau(V) &\geq \alpha \cdot \tau(V) + \frac{n}{K} && \text{(Since } \tau(V) \geq n\text{)} \\ &\geq \alpha \cdot \max\{2n - \rho(G), n + \beta - 2\} + \frac{n}{K} && \text{(By Claim 8.2 and Lemma 8.3)} \\ &\geq \alpha \cdot \max\{2n - \rho(G), n + \beta\} + \frac{n}{K} - 4.\end{aligned}$$

So in order to show that $\tilde{\tau} \leq (\alpha + \frac{1}{K}) \cdot \tau(V)$, it is sufficient to show that

$$2n - \frac{1}{5}(\frac{\rho(G)}{2} - 2\beta) + \frac{4n}{5K} \leq \alpha \cdot \max\{2n - \rho(G), n + \beta\} + \frac{n}{K} - 4.$$

If n is large enough, then we have $\frac{4n}{5K} \leq \frac{n}{K} - 4$, which implies that we need to prove

$$2n - \frac{1}{5}(\frac{\rho(G)}{2} - 2\beta) \leq \alpha \cdot \max\{2n - \rho(G), n + \beta\}.$$

Now, let $\rho(G) = xn$ and $\beta = yn$ for $0 \leq x \leq 1$ and $0 \leq y \leq 1$. To obtain α, it suffices to solve the following program

$$\begin{aligned}\text{maximize} \quad & \alpha \\ \text{subject to} \quad & \frac{2 - \frac{1}{5}(\frac{x}{2} - 2y)}{\max\{2-x, 1+y\}} \leq \alpha, \\ & 0 \leq x \leq 1, \\ & 0 \leq y \leq 1.\end{aligned}$$

This is a constant size program that can be easily solved; the solution is 19/10.[4] This completes the proof. □

[4] See e.g. this WolframAlpha link.

Theorem 8.9. Let $\tau(V)$ be the cost of graphic TSP of graph $G = (V, E)$. For any $\varepsilon > 0$, there exists an algorithm that estimate the cost of graphic TSP, $\tilde{\tau}$, such that

$$\tau(V) \leq \tilde{\tau} \leq (\frac{19}{10} + \varepsilon) \cdot \tau(V),$$

w.h.p in $\tilde{O}(n)$ running time.

Proof. Let $\tilde{\tau}$ be the output of Algorithm 7. If we choose $K = \frac{1}{\varepsilon}$, then by Lemma 8.8, we have

$$\tau(V) \leq \tilde{\tau} \leq (\frac{19}{10} + \varepsilon) \cdot \tau(V).$$

Also, by Theorem 6.8 and Lemma 8.7, estimating $\tilde{\rho}$ and $\tilde{\beta}$ can be done in $\tilde{O}(n)$ time. □

9 Further Improvement for Graphic TSP

In this section, we design an algorithm to estimate the number of bridges in given graph G. Equipped with this tool, we are able to estimate the number of non-bridge edges in the path cover which helps to improve the approximation ratio. Before describing the techniques for estimating the number of bridges, we prove the following lemma that provides a lower bound on the cost of graphic TSP based on the number of bridges in the graph.

Claim 9.1. If the number of bridges in the graph G is at least B, then the cost of the graphic TSP is at least $n + B$.

Proof. Since the metric in the graphic TSP is corresponding to the shortest path distances in graph G, then a TSP tour is corresponding to a closed walk that contains all vertices. Thus, each bridge should be crossed at least two times in this walk in order for the walk to be closed and cover all vertices. Therefore, the cost of graphic TSP is at least $n + B$. □

In the following series of lemmas, first, we prove that there are a few bridges that both of their endpoints have a high degree and then we show an efficient way to estimate the number of bridges that have at least one endpoint with a low degree. Combining the above arguments is the main idea to estimate the number of bridges.

Lemma 9.2. For any integer $c \geq 2$, there exists at most $\frac{2n}{c}$ bridges that both of their endpoints have a degree larger than c.

Proof. Let $\mathcal{B}$ be the set of bridges that both of their endpoints have a degree larger than c. We construct a tree, $T_{\mathcal{B}}$, with edge set equal to $\mathcal{B}$ such that each vertex of $T_{\mathcal{B}}$ corresponds to a component of vertices that are compressed to a single vertex. We construct $T_{\mathcal{B}}$ iteratively. In the beginning, we consider the bridge-block tree of the original graph. In each step, if there exists a bridge $e = (u, v)$ (note that u and v are vertices of the tree and corresponding to a set of vertices of the original graph) such that at least one of its endpoints has a degree less than or equal to c, we merge u with v and add all edges of u to v. We continue this process until there is no bridge with an endpoint of degree less than or equal to c.

Now, we prove that $|\mathcal{B}| \leq \frac{2n}{c}$. Let x_v denote the number of vertices in the original graph that are compressed to vertex $v \in T_{\mathcal{B}}$. We remove vertices of $T_{\mathcal{B}}$ one by one until there is no vertex in the tree. At each step, we remove a leaf $v \in T_{\mathcal{B}}$ and at the end when only one vertex is remaining,

we remove that vertex. Let y_v be the number of incident edges to v in $T_{\mathcal{B}}$ that are removed before removing v. At the time that we are removing leaf v, we have $x_v + y_v + 1 \geq c+1$, since the endpoint of the leaf that is the component of v has at most x_v incident edges in the same component in the original graph, y_v incident edges to the other components that are removed before, and there is only one remaining incident edge to other components (the other endpoint of the leaf). Thus,

$$\sum_{v \in T_{\mathcal{B}}} x_v \geq \sum_{v \in T_{\mathcal{B}}} (c - y_v) = (|\mathcal{B}| + 1)c - \sum_{v \in T_{\mathcal{B}}} y_v. \tag{4}$$

Since vertices of each component are disjoint, we have $\sum_{v \in T_{\mathcal{B}}} x_v = n$. Moreover, we have $\sum_{v \in T_{\mathcal{B}}} y_v = |\mathcal{B}|$ since each edge of $\mathcal{B}$ counted when one of its endpoints is deleted from the tree. Combining above bounds and inequality (4), we have

$$n = \sum_{v \in T_{\mathcal{B}}} x_v \geq (|\mathcal{B}| + 1)c - |\mathcal{B}|$$

Therefore,

$$|\mathcal{B}| \leq \frac{n - c}{c - 1} \leq \frac{2n}{c},$$

where the last inequality holds for sufficiently large n. □

Lemma 9.3. Let $c \geq 2$ be a constant and u is a vertex such that $\deg(u) \leq c$. Then we can test if each of incident edges of u is a bridge in $O(n)$ total running time.

Proof. We can query all neighbors of u in $O(n)$. Assume that $\{v_1, v_2, \ldots, v_r\}$ are neighbors of u for $r \leq c$. Now we divide the vertices of the graph except u into r sets $V_1, V_2, \ldots, V_r$. For each vertex $w \neq u$, we query the distance of w to all $\{v_1, v_2, \ldots, v_r\}$. Let v_i be the closest one to w (if there is a tie, choose the one with the lowest index). Then we put w in V_i. Note that since c is a constant and $r \leq c$, this step can be done in $O(n)$.

Now we claim that (u, v_j) is a bridge iff the following conditions hold:

- For each $w \in V_j$ and $i \neq j$, $d(w, v_i) - d(w, v_j) = 2$.
- For each $w \in V_i$ such that $i \neq j$, $d(w, v_j) - d(w, v_i) = 2$.

Suppose that $e = (u, v_j)$ is a bridge. Since removing e creates two connected components C_u and C_{v_j}, all vertices in C_{v_j} (resp. C_u) have a closer distance to v_j (resp. u). In other words, all shortest paths between $w \in V_j$ to v_i for $i \neq j$, cross edges (v_j, u) and (u, v_i). In addition, all the shortest paths between $w \in V_i$ and v_j for $i \neq j$, cross edges (v_j, u) and (u, v_i). Therefore, both conditions hold.

Now suppose that $e = (u, v_j)$ is not a bridge. In this case, there must be an edge between V_j and at least one of V_i as otherwise, V_j will be disconnected from the rest of the graph by removing e. Without loss of generality, assume that this edge is (w, w') such that $w \in V_j$, $w' \in V_i$, and $i \neq j$. Also, w.l.o.g., we assume $d(w, v_j) \leq d(w', v_i)$. Since there is an edge between w and w', we have $d(w', v_j) \leq 1 + d(w, v_j) \leq 1 + d(w', v_i)$, which contradicts the conditions.

To test whether the conditions hold, we need to query the distance of each vertex to all $\{v_0, v_1, \ldots, v_r\}$ which can be done in $O(n)$ in total since r is a constant. □

Lemma 9.4. Let B be the number of bridges in graph G. For any $\varepsilon > 0$, there exists an algorithm that outputs an estimate $\tilde{B}$ in $\tilde{O}(n)$ such that $B \leq \tilde{B} \leq B + \varepsilon n$.

Proof. By Lemma 9.2, there are at most $\frac{\varepsilon n}{2}$ bridges with both endpoints have degree larger than $\frac{4}{\varepsilon}$. Let $\hat{B}$ be the number of bridges that at least one of their endpoint has degree of at most $\frac{4}{\varepsilon}$. Thus,

$$B - \frac{\varepsilon n}{2} \leq \hat{B} \leq B. \tag{5}$$

We sample $r = 256 \cdot \varepsilon^{-4} \cdot \log n$ vertices uniformly at random with replacement. Let u be the i-th sampled vertex. If the degree of the vertex is larger than $\frac{4}{\varepsilon}$, we let $X_i = 0$. Otherwise, let $\{v_1, v_2, \ldots, v_k\}$ be the neighbors of u where $k \leq \frac{4}{\varepsilon}$. By Lemma 9.3, we can test if each of the incident edges of u is a bridge in $O(n)$ total running time. For each edge (u, v_j) if $\deg(u) < \deg(v_j)$ or $\deg(u) = \deg(v_j)$ and index of u is smaller than v_j, we test if the edge is a bridge or not. Let X_i show the number of successful tests for incident edges of u. Note that in the above algorithm, each bridge with low-degree endpoints only counted once.

Let $\bar{X} = (\sum_i^r X_i)/r$ and $n\bar{X} + \frac{3\varepsilon n}{4}$ be our final estimate of the number of bridges. Hence, $\mathrm{E}[\bar{X}] = \hat{B}/n$. Since $\bar{X}$ is the average of r independent random variables such that $0 \leq X_i \leq 4/\varepsilon$, by Hoeffding's inequality (Proposition 3.3) we obtain

$$\Pr\left[\left|\bar{X} - \mathrm{E}[\bar{X}]\right| \geq \frac{\varepsilon}{4}\right] \leq 2\exp\left(-\frac{r\varepsilon^4}{128}\right) = \frac{2}{n^2},$$

where the last inequality follows from $r = 256 \cdot \varepsilon^{-4} \cdot \log n$. Therefore, with probability of $1 - \frac{2}{n^2}$,

$$\begin{aligned} n\bar{X} &\in n\,\mathrm{E}[\bar{X}] \pm \frac{n\varepsilon}{4} \\ &= \hat{B} \pm \frac{n\varepsilon}{4} && (\text{Since } \mathrm{E}[\bar{X}] = \hat{B}/n). \end{aligned}$$

Combining above range and inequality (5), we get

$$B \leq n\bar{X} + \frac{3\varepsilon n}{4} \leq B + \varepsilon n.$$

Since the number of sampled vertices is $r = 256 \cdot \varepsilon^{-4} \cdot \log n$, the total running time is $\tilde{O}(n)$. □

Now we are ready to introduce the improved algorithm for graphic TSP.

Algorithm 8: Improved algorithm for graphic TSP.

1 Construct $\hat{G} = (V_{\hat{G}}, E_{\hat{G}})$ implicitly as described in Section 6.
2 Let $\tilde{\rho}$ be the output of Algorithm 5 on $\hat{G}$.
3 Let $\tilde{B}$ be the estimate of the number of bridges.
4 $\tilde{\tau} = 2n - \frac{1}{3}(\tilde{\rho} - \tilde{B})$
5 return $\tilde{\tau}$

Lemma 9.5. Let $\tilde{\tau}$ be the output of Algorithm 8 and $\tau(V)$ be the cost of graphic TSP of graph $G = (V, E)$. With high probability,

$$\tau(V) \leq \tilde{\tau} \leq (\frac{11}{6} + \frac{1}{K}) \cdot \tau(V),$$

where K is the parameter which is defined in Algorithm 2.

Proof. Let $\rho(G)$ be the size of maximum path cover and B be the number of bridges in the graph. There are at least $\rho(G)-B$ edges of maximum path cover that are not bridge. These edges construct disjoint paths which implies there exists a matching of size $\frac{1}{2}(\rho(G)-B)$ that none of its edges is a bridge. Hence, by Lemma 8.5, the cost of graphic TSP is at most $2n-\frac{1}{3}(\rho(G)-B)$. Therefore, since $\tilde{\rho} \leq \rho(G)$ and $B \leq \tilde{B}$, we get $\tau(V) \leq \tilde{\tau}$.

By Lemma 6.7 and Lemma 9.4, we have $\left(\frac{1}{2}-\frac{1}{K}\right)\cdot\rho(G)-\frac{n}{K} \leq \tilde{\rho}$ and $\tilde{B} \leq B+\frac{n}{K}$ which implies

$$\begin{aligned}\tilde{\tau} &\leq 2n-\frac{1}{3}\left((\frac{1}{2}-\frac{1}{K})\cdot\rho(G)-(B+\frac{n}{K})\right)\\ &\leq 2n-\frac{1}{3}(\frac{\rho(G)}{2}-B)+\frac{2n}{K} && \text{(Since } \rho(G)\leq n\text{).}\end{aligned}$$

Also, assume that the approximation ratio that the algorithm obtains is $\alpha+2/K$ for some $\alpha \leq 2$. Thus,

$$\begin{aligned}(\alpha+\frac{2}{K})\cdot\tau(V) &\geq \alpha\cdot\tau(V)+\frac{2n}{K} && \text{(Since } \tau(V)\geq n\text{)}\\ &\geq \alpha\cdot\max\{2n-\rho(G),n+B\}+\frac{2n}{K} && \text{(By Claim 8.2 and Claim 9.1).}\end{aligned}$$

Therefore, to show that $(\alpha+\frac{1}{K})\cdot\tau(V)\geq\tilde{\tau}$, it is sufficient to show

$$2n-\frac{1}{3}(\frac{\rho(G)}{2}-B) \leq \alpha\cdot\max\{2n-\rho(G),n+B\}.$$

Now, let $\rho(G)=xn$ and $B=yn$ for $0\leq x\leq 1$ and $0\leq y\leq 1$. To obtain α, we write the following maximization problem,

$$\begin{aligned}\text{maximize}\quad & \alpha\\ \text{subject to}\quad & \tfrac{2-\frac{1}{3}(\frac{x}{2}-y)}{\max\{2-x,1+y\}}\leq\alpha,\\ & 0\leq x\leq 1,\\ & 0\leq y\leq 1.\end{aligned}$$

The solution to this problem is $11/6$.[5] This completes the proof. □

Theorem 9.6. Let $\tau(V)$ be the cost of graphic TSP of graph $G=(V,E)$. For any $\varepsilon>0$, there exists an algorithm that estimate the cost of graphic TSP, $\tilde{\tau}$, such that

$$\tau(V)\leq\tilde{\tau}\leq(\frac{11}{6}+\varepsilon)\cdot\tau(V),$$

w.h.p in $\tilde{O}(n)$ running time.

Proof. Let $\tilde{\tau}$ be the output of Algorithm 8. If we choose $K=\frac{1}{\varepsilon}$, then by Lemma 9.5, we have

$$\tau(V)\leq\tilde{\tau}\leq(\frac{11}{6}+\varepsilon)\cdot\tau(V).$$

Also, by Theorem 6.8 and Lemma 9.4, estimating $\tilde{\rho}$ and $\tilde{B}$ can be done in $\tilde{O}(n)$ time. □

[5] See e.g. this WolframAlpha link.

10 A Slightly Subquadratic Algorithm for Graphic TSP

With the recent advances in designing sublinear algorithms for maximum matching [4, 5, 9, 8], we can now achieve a more precise estimation of the size of graphic TSP at the cost of increased running time. In this section, we present an algorithm that approximates the graphic TSP with an accuracy within a factor of $5/3 + \varepsilon$ in $O(n^{2-\Omega_\varepsilon(1)})$. We use the following result by Bhattacharya, Kiss, and Saranurak [8] to design our slightly subquadratic graphic TSP estimator.

Proposition 10.1. Suppose that we have access to the adjacency matrix of graph G. Then, There exists an algorithm that estimates the size of the maximum matching of graph G, $\tilde{\mu}$, such that

$$\mu(G) - \varepsilon n \leq \tilde{\mu} \leq \mu(G),$$

w.h.p in $n^{2-\Omega_\varepsilon(1)}$ time.

Combining this algorithm with the framework of Chen, Kannan, and Khanna [11] implies a $(13/7 + \varepsilon)$-approximation for graphic TSP in $n^{2-\Omega_\varepsilon(1)}$ time. Our algorithm in Section 9 makes an improvement on both the running time and approximation ratio for the graphic TSP over this recent result. Furthermore, using Proposition 10.1 and our estimator for counting the number of bridges, we are able to obtain a 5/3-approximation ratio. Now we are ready to propose our algorithm.

Algorithm 9: Slightly subquadratic algorithm for graphic TSP.

1 Let $\tilde{\mu}$ be the output of Proposition 10.1 on G.
2 Let $\tilde{B}$ be the estimate of the number of bridges.
3 $\tilde{\tau} = 2n - \frac{1}{3}(\tilde{\mu} - \tilde{B})$
4 return $\tilde{\tau}$

Lemma 10.2. Let $\tilde{\tau}$ be the output of Algorithm 9 and $\tau(V)$ be the cost of graphic TSP of graph $G = (V, E)$. With high probability,

$$\tau(V) \leq \tilde{\tau} \leq (\frac{5}{3} + \varepsilon) \cdot \tau(V).$$

Proof. Let B be the number of bridges in G. There exists a matching with a size of at least $\mu(G) - B$ that none of its edges is a bridge. Thus, by Lemma 8.5, it holds $\tau(V) \leq 2n - \frac{1}{3}(\mu(G) - B)$. Combining with $\tilde{\mu} \leq \mu(G)$ and $B \leq \tilde{B}$, we get $\tau(V) \leq \tilde{\tau}$.

Additionally, by Proposition 10.1 and Lemma 9.4, we have $\tilde{\mu} \geq \mu(G) - \varepsilon n$ and $\tilde{B} \leq B + \varepsilon n$. Thus,

$$\begin{aligned}
\tilde{\tau} &\leq 2n - \frac{1}{3}\left((\mu(G) - \varepsilon n) - (B + \varepsilon n)\right) \\
&\leq 2n - \frac{1}{3}(\mu(G) - B) + 2\varepsilon n && \text{(Since } \mu(G) \leq n\text{).}
\end{aligned}$$

Assume the approximation ratio of the algorithm is $\alpha + 2\varepsilon$ for some $\alpha \leq 2$, we must have

$$\begin{aligned}
(\alpha + 2\varepsilon) \cdot \tau(V) &\geq \alpha \cdot \tau(V) + 2\varepsilon n && \text{(Since } \tau(V) \geq n\text{)} \\
&\geq \alpha \cdot \max\{2n - \mu(G), n + B\} + 2\varepsilon n. && \text{(By Lemma 8.4 and Claim 9.1)}
\end{aligned}$$

In order to show $(\alpha + 2\varepsilon) \cdot \tau(V) \geq \tilde{\tau}$, it is sufficient to show

$$2n - \frac{1}{3}(\mu(G) - B) \leq \alpha \cdot \max\{2n - \mu(G), n + B\}.$$

Let $\mu(G) = xn$ and $B = yn$ for $0 \leq x \leq 1$ and $0 \leq y \leq 1$. To obtain α, we write the following maximization problem,

$$\begin{aligned} \text{maximize} \quad & \alpha \\ \text{subject to} \quad & \tfrac{2-\frac{1}{3}(x-y)}{\max\{2-x,1+y\}} \leq \alpha, \\ & 0 \leq x \leq 1, \\ & 0 \leq y \leq 1. \end{aligned}$$

The solution to this problem is 5/3.[6] □

Theorem 10.3. Let $\tau(V)$ be the cost of graphic TSP of graph $G = (V, E)$. For any $\varepsilon > 0$, there exists an algorithm that estimate the cost of graphic TSP, $\tilde{\tau}$, such that

$$\tau(V) \leq \tilde{\tau} \leq (\frac{5}{3} + \varepsilon) \cdot \tau(V),$$

w.h.p in $n^{2-\Omega_\varepsilon(1)}$ running time.

Proof. Let $\tilde{\tau}$ be the output of Algorithm 9. By Lemma 9.5, we have

$$\tau(V) \leq \tilde{\tau} \leq (\frac{5}{3} + \varepsilon) \cdot \tau(V).$$

Moreover, by Proposition 10.1 and Lemma 9.4, estimating $\tilde{\mu}$ and $\tilde{B}$ can be done in $n^{2-\Omega_\varepsilon(1)}$ time. □

11 Lower Bound for Approximating Maximum Path Cover

11.1 "Conditional" Hardness for the Approximation Ratio

In this section, we prove that if there exists a constant $\alpha > 0$ and an algorithm that returns a $(\frac{1}{2} + \alpha)$-approximate estimate for the size of maximum path cover in $\tilde{O}(n)$ time in a bipartite graph, then there is a $(\frac{1}{2} + \alpha)$-approximate algorithm for estimating the maximum matching size in $\tilde{O}(n)$ time. This remains an important open problem in the study of sublinear time maximum matching algorithms. See in particular [5]. This implies that short of a major result in the study maximum matchings in the sublinear time model, which have received significant attention in the literature (see [30, 2, 5, 4, 9] and references therein), our path cover algorithm has an optimal approximation ratio.

Let $G = (V, U, E)$ be a bipartite graph. We construct a graph $G' = (V', U', E')$ such that a better than $\frac{1}{2}$-approximate estimate of maximum path cover on G' leads to a better than $\frac{1}{2}$-approximate estimate of maximum matching in G. Let r be a large constant. We create r copies of G, showing the i-th copy with $G_i = (V_i, U_i, E)$. Also, we create another $r - 1$ copies $H_1, \ldots, H_{r-1}$ of G with $H_i = (V_i, U_{i+1}, E)$. Now we let the $G' = (\bigcup_{i=1}^{r} G_i) \cup (\bigcup_{i=1}^{r-1} H_i)$. Now we claim that the size of maximum path cover of the graph G' is roughly $2r \cdot \mu(G)$ which can be used as an estimator for the maximum matching of G.

[6] See e.g. this WolframAlpha link.

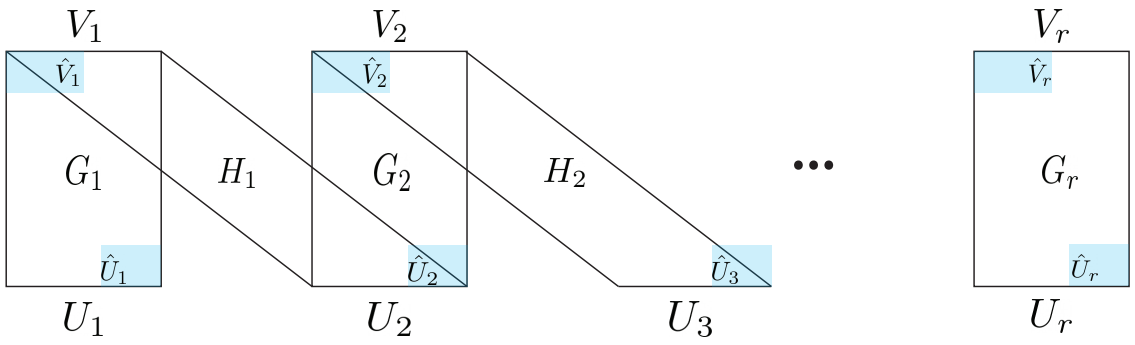


Figure 3: Illustration of graph $G' = (V', U', E')$. Each G_i is shown by a rectangle and each H_i is shown by a parallelogram. Top and bottom horizontal lines illustrate V_i and U_i. Blue highlighted parts represent the vertex cover of the graph.

Before proving the main result of this section, we characterize some properties of the constructed graph G'.

Claim 11.1. $\mu(G') = r \cdot \mu(G)$.

Proof. First, since all graphs $\{G_i\}_{i=1}^r$ are the same as G and are vertex-disjoint, if we consider the maximum matching of G in each of the r graphs, we will have a matching of size $r \cdot \mu(G)$. Thus, $\mu(G') \geq r \cdot \mu(G)$.

Let $\hat{V} \cup \hat{U}$ be the minimum vertex cover of G such that $\hat{V} \in V$ and $\hat{U} \in U$. By König's Theorem (Proposition 3.1), we have $|\hat{V} \cup \hat{U}| = \mu(G)$. Now we show there exists a vertex cover of size $r \cdot \mu(G)$ for graph G'. Let $\hat{V}_i \in V_i$ (resp. $\hat{U}_i \in U_i$) be the copy of vertices V (resp. U) in graph G_i. We claim $(\bigcup_{i=1}^r \hat{V}_i \cup \hat{U}_i)$ is a vertex cover for G'. If an edge is in G_i, then at least one of its endpoints is in $\hat{V}_i \cup \hat{U}_i$ since $\hat{V}_i \cup \hat{U}_i$ is a vertex cover of G_i. Moreover, by the construction, $\hat{V}_i \cup \hat{U}_{i+1}$ is a vertex cover of H_i. Hence, each edge of H_i is also covered by the vertex cover. Therefore, since there exists a vertex cover of size $|(\bigcup_{i=1}^r \hat{V}_i \cup \hat{U}_i)| = r \cdot |\hat{V} \cup \hat{U}| = r \cdot \mu(G)$, then we have $\mu(G') \leq r \cdot \mu(G)$ which completes the proof. □

Observation 11.2. It holds $(2r-1) \cdot \mu(G) \leq \rho(G') \leq 2r \cdot \mu(G)$.

Proof. Since the union of maximum matching of all graphs $\{G_i\}_{i=1}^r$ and $\{H_i\}_{i=1}^{r-1}$ creates a path cover, we get $(2r-1) \cdot \mu(G) \leq \rho(G')$. Futhermore, if there exists a path cover of size larger than $2r \cdot \mu(G)$, then the maximum matching of these paths will be larger than $r \cdot \mu(G)$ which contradicts Claim 11.1. Thus, $\rho(G') \leq 2r \cdot \mu(G)$. □

Now we are ready to show the reduction.

Lemma 11.3. For any constant $\alpha > 0$, if there exists an algorithm that can estimate the maximum path cover within a $(\frac{1}{2}+\alpha)$-factor in $O(T(n))$ time, then the same algorithm can be used to estimate the maximum matching of bipartite graph G within a $(1-\varepsilon) \cdot (\frac{1}{2}+\alpha)$-factor in $O(T(n/\varepsilon))$ time.

Proof. We construct graph G' as described at the beginning of the section with $r = \frac{1}{2\varepsilon}$. By Observation 11.2, $(\frac{1}{\varepsilon}-1) \cdot \mu(G) \leq \rho(G') \leq \frac{1}{\varepsilon} \cdot \mu(G)$. Let $\tilde{\rho}$ be the estimate of the algorithm for the maximum path cover of G'. Hence, we have

$$(\frac{1}{2}+\alpha)(\frac{1}{\varepsilon}-1) \cdot \mu(G) \leq \tilde{\rho} \leq \frac{1}{\varepsilon} \cdot \mu(G).$$

Now let $\tilde{\mu} = \varepsilon \cdot \tilde{\rho}$ be the estimate for the maximum matching of G. Hence,

$$(1-\varepsilon)(\frac{1}{2}+\alpha)\cdot\mu(G) \leq \tilde{\mu} \leq \mu(G).$$

Since the number of vertices and number of edges of G' is $r = \frac{1}{2\varepsilon}$ times more than G, then the running time will be $O(T(n/\varepsilon))$. □

A reduction to matchings can also be proved for $(1,2)$-TSP, albeit with an extra promise for the matching instance that the matching is either perfect or half-perfect. This problem, formalized below, also remains open in the study matchings. We show that a better than 1.5-approximation for $(1,2)$-TSP in $\widetilde{O}(n)$ time would resolve this question.

Problem 11.4. Suppose that we are given a bipartite graph $G = (L, R, E)$ with $|L| = |R| = n$ and are promised that either $\mu(G) = n$ or $\mu(G) = (\frac{1}{2}+\varepsilon)n/2$ for any desirably small constant $\varepsilon > 0$. Provided adjacency matrix access to the graph, does there exist an $n^{1+o(1)}$ time algorithm that distinguishes the two?

Theorem 11.5. If there is an algorithm that estimates the size of $(1,2)$-TSP within a $(\frac{3}{2}-\varepsilon_0)$-factor for some fixed constant $\varepsilon_0 \in (0, \frac{1}{4}]$ in $n^{1+o(1)}$, then Problem 11.4 can indeed be solved in $n^{1+o(1)}$ time.

Proof. Let G_1 and G_2 be two graphs with n vertices such that $\mu(G_1) = n$ and $\mu(G_2) = (\frac{1}{2} + \frac{\varepsilon_0}{16})$. We construct graph $G'_1 = (V'_1, E'_1)$ and $G'_2 = (V'_2, E'_2)$ as described at the beginning of the section with $r = \frac{1}{\varepsilon_0}$. By Observation 11.2, we have $\rho(G'_1) \geq (\frac{2}{\varepsilon_0} - 1)n$ and $\rho(G'_2) \leq (\frac{1}{\varepsilon_0} + \frac{1}{8})n$. Thus, by Observation 7.1, we get

$$\tau(V'_1) \leq \frac{4}{\varepsilon_0}n - (\frac{2}{\varepsilon_0} - 1)n = (\frac{2}{\varepsilon_0} + 1)n,$$
$$\tau(V'_2) \geq \frac{4}{\varepsilon_0}n - (\frac{1}{\varepsilon_0} + \frac{1}{8})n - 1 \geq (\frac{3}{\varepsilon_0} - \frac{1}{4})n,$$

for sufficiently large n, which implies

$$\frac{\tau(V'_2)}{\tau(V_1)} = \frac{3-\varepsilon_0/4}{2+\varepsilon_0} \geq \frac{3}{2} - \varepsilon_0.$$

Therefore, the algorithm for (1,2)-TSP can distinguish between G'_1 and G'_2 which implies Problem 11.4 can be solved in $n^{1+o(1)}$ time for $\varepsilon = \varepsilon_0/16$. □

Lower Bound for Graphic TSP: note that we can reduce an instance of (1,2)-TSP to graphic TSP by adding a new vertex and connecting the newly added vertex to all vertices of the graph. Therefore, the $n^{1+o(1)}$ time lower bound also holds for graphic TSP.

11.2 Information-Theoretic Lower Bounds on the Running Time

Since any constant approximation algorithm for estimating maximum path cover can be used to estimate the size of matching within a constant factor, then all of the lower bounds for $O(1)$-approximating maximum matching in sublinear time also hold for (1)-approximating maximum path cover in sublinear time. We restate some of these lower bounds along with a short proof (see [2] for a detailed discussion).

Lemma 11.6. Any algorithm that estimates maximum path cover within a constant multiplicative factor requires $\Omega(n)$ queries in the adjacency list model.

Proof. Consider two graphs that the first one does not have any edge and the second one has only a single edge. In order to give any multiplicative approximation for maximum path cover, the algorithm needs to find the edge which requires $\Omega(n)$ queries in the adjacency list model. □

Lemma 11.7. Any algorithm that estimates maximum path cover within a constant multiplicative factor require $\Omega(n^2)$ queries in the adjacency matrix model.

Proof. Consider the same construction as Lemma 11.6. To give any multiplicative approximation for maximum path cover, the algorithm needs to find the edge which requires $\Omega(n^2)$ queries in the adjacency matrix model. □

Lemma 11.8. Any algorithm that estimates maximum path cover within a multiplicative-additive requires $\Omega(n)$ queries in the adjacency matrix model.

Proof. Consider a graph with no edge and a graph with one Hamiltonian cycle and no other edges. In order for the algorithm to distinguish between these two graphs, it must find at least one edge of the second graph which requires $\Omega(n)$ queries in the adjacency matrix model. □

There is also a lower bound for multiplicative-additive estimation of matching in adjacency list model [27] that also holds for maximum path cover.

Lemma 11.9. Any algorithm that estimates maximum path cover within a constant multiplicative-additive factor requires $\Omega(\bar{d})$ queries in the adjacency list model.

Acknowledgements. Mohammad Roghani and Amin Saberi were supported by NSF award 1812919 and ONR award 141912550. Soheil Behnezhad and Aviad Rubinstein were supported by NSF CCF-1954927, and a David and Lucile Packard Fellowship. Soheil Behnezhad was additionally supported by NSF Awards 1942123, 1812919 and by Moses Charikar's Simons Investigator Award.

References

[1] Anna Adamaszek, Matthias Mnich, and Katarzyna Paluch. New approximation algorithms for (1, 2)-tsp. In 45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, July 9-13, 2018, Prague, Czech Republic, volume 107 of LIPIcs, pages 9:1–9:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[2] Soheil Behnezhad. Time-Optimal Sublinear Algorithms for Matching and Vertex Cover. In 62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022, pages 873–884. IEEE, 2021.

[3] Soheil Behnezhad, Mohammad Roghani, and Aviad Rubinstein. Local computation algorithms for maximum matching: New lower bounds. In 64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023, pages 2322–2335. IEEE, 2023. doi: 10.1109/FOCS57990.2023.00143. URL https://doi.org/10.1109/FOCS57990.2023.00143.

[4] Soheil Behnezhad, Mohammad Roghani, and Aviad Rubinstein. Sublinear time algorithms and complexity of approximate maximum matching. Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023, pages 267–280, 2023. doi: 10.1145/3564246.3585231. URL https://doi.org/10.1145/3564246.3585231.

[5] Soheil Behnezhad, Mohammad Roghani, Aviad Rubinstein, and Amin Saberi. Beating greedy matching in sublinear time. In Nikhil Bansal and Viswanath Nagarajan, editors, Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023, pages 3900–3945. SIAM, 2023. doi: 10.1137/1.9781611977554.CH151. URL https://doi.org/10.1137/1.9781611977554.ch151.

[6] Soheil Behnezhad, Mohammad Roghani, and Aviad Rubinstein. Approximating maximum matching requires almost quadratic time. Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, Canada, To Appear, 2024.

[7] Piotr Berman and Marek Karpinski. 8/7-approximation algorithm for (1, 2)-tsp. In Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2006, Miami, Florida, USA, January 22-26, 2006, pages 641–648. ACM Press, 2006.

[8] Sayan Bhattacharya, Peter Kiss, and Thatchaphol Saranurak. Dynamic $(1+\varepsilon)$-approximate matching size in truly sublinear update time. 64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023, pages 1563–1588, 2023. doi: 10.1109/FOCS57990.2023.00095. URL https://doi.org/10.1109/FOCS57990.2023.00095.

[9] Sayan Bhattacharya, Peter Kiss, and Thatchaphol Saranurak. Sublinear algorithms for $(1.5+\varepsilon)$-approximate matching. Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023, pages 254–266, 2023. doi: 10.1145/3564246.3585252. URL https://doi.org/10.1145/3564246.3585252.

[10] Guy E. Blelloch, Jeremy T. Fineman, and Julian Shun. Greedy Sequential Maximal Independent Set and Matching are Parallel on Average. In 24th ACM Symposium on Parallelism in Algorithms and Architectures, SPAA '12, Pittsburgh, PA, USA, June 25-27, 2012, pages 308–317. ACM, 2012.

[11] Yu Chen, Sampath Kannan, and Sanjeev Khanna. Sublinear Algorithms and Lower Bounds for Metric TSP Cost Estimation. In 47th International Colloquium on Automata, Languages, and Programming (ICALP 2020), volume 168 of Leibniz International Proceedings in Informatics (LIPIcs), pages 30:1–30:19, Dagstuhl, Germany, 2020. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. ISBN 978-3-95977-138-2.

[12] Yu Chen, Sanjeev Khanna, and Zihan Tan. Sublinear algorithms and lower bounds for estimating MST and TSP cost in general metrics. 50th International Colloquium on Automata, Languages, and Programming, ICALP 2023, July 10-14, 2023, Paderborn, Germany, 261:37:1–37:16, 2023. doi: 10.4230/LIPICS.ICALP.2023.37. URL https://doi.org/10.4230/LIPIcs.ICALP.2023.37.

[13] Nicos Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Technical report, Carnegie-Mellon Univ Pittsburgh Pa Management Sciences Research Group, 1976.

[14] Artur Czumaj and Christian Sohler. Estimating the weight of metric minimum spanning trees in sublinear-time. In Proceedings of the 36th Annual ACM Symposium on Theory of Computing, Chicago, IL, USA, June 13-16, 2004, pages 175–183, 2004.

[15] Artur Czumaj and Christian Sohler. Estimating the weight of metric minimum spanning trees in sublinear time. SIAM J. Comput., 39(3):904–922, 2009.

[16] Shayan Oveis Gharan, Amin Saberi, and Mohit Singh. A randomized rounding approach to the traveling salesman problem. In IEEE 52nd Annual Symposium on Foundations of Computer Science, FOCS 2011, Palm Springs, CA, USA, October 22-25, 2011, pages 550–559. IEEE Computer Society, 2011.

[17] Anupam Gupta and Krzysztof Onak. Sublinear.info Open Problem 71: Metric TSP Cost Approximation. 2016. Available at https://sublinear.info/index.php?title=Open_Problems:71.

[18] Anna R. Karlin, Nathan Klein, and Shayan Oveis Gharan. A (slightly) improved approximation algorithm for metric TSP. In STOC '21: 53rd Annual ACM SIGACT Symposium on Theory of Computing, Virtual Event, Italy, June 21-25, 2021, pages 32–45. ACM, 2021.

[19] Richard M Karp. Reducibility among combinatorial problems. In Complexity of computer computations, pages 85–103. Springer, 1972.

[20] Marek Karpinski and Richard Schmied. On approximation lower bounds for TSP with bounded metrics. Electron. Colloquium Comput. Complex., page 8, 2012.

[21] D. König. Über graphen und ihre anwendung auf determinantentheorie und mengenlehre. Mathematische Annalen, 77:453–465, 1916. URL http://eudml.org/doc/158740.

[22] Matthias Mnich and Tobias Mömke. Improved integrality gap upper bounds for traveling salesperson problems with distances one and two. Eur. J. Oper. Res., 266(2):436–457, 2018.

[23] Tobias Mömke and Ola Svensson. Approximating graphic TSP by matchings. In IEEE 52nd Annual Symposium on Foundations of Computer Science, FOCS 2011, Palm Springs, CA, USA, October 22-25, 2011, pages 560–569. IEEE Computer Society, 2011.

[24] Marcin Mucha. 13/9-approximation for graphic TSP. In 29th International Symposium on Theoretical Aspects of Computer Science, STACS 2012, February 29th - March 3rd, 2012, Paris, France, volume 14 of LIPIcs, pages 30–41. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2012.

[25] Krzysztof Onak, Dana Ron, Michal Rosen, and Ronitt Rubinfeld. A near-optimal sublinear-time algorithm for approximating the minimum vertex cover size. In Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2012, Kyoto, Japan, January 17-19, 2012, pages 1123–1131. SIAM, 2012.

[26] Christos H. Papadimitriou and Mihalis Yannakakis. The traveling salesman problem with distances one and two. Math. Oper. Res., 18(1):1–11, 1993.

[27] Michal Parnas and Dana Ron. Approximating the minimum vertex cover in sublinear time and a connection to distributed algorithms. Theoretical Computer Science, 381(1):183–196, 2007. ISSN 0304-3975.

[28] András Sebö and Jens Vygen. Shorter tours by nicer ears: 7/5-approximation for the graph-tsp, 3/2 for the path version, and 4/3 for two-edge-connected subgraphs. Comb., 34(5):597–629, 2014.

[29] Anatoliy I Serdyukov. O nekotorykh ekstremal'nykh obkhodakh v grafakh. Upravlyayemyye sistemy, 17:76–79, 1978.

[30] Yuichi Yoshida, Masaki Yamamoto, and Hiro Ito. An improved constant-time approximation algorithm for maximum matchings. In Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009, Bethesda, MD, USA, May 31 - June 2, 2009, pages 225–234. ACM, 2009.

A Implementation Details

In this section, we discuss why Lemma 6.1, restated below, holds.

Lemma 6.1. There exists a data structure that given a graph G in the adjacency list format, (implicitly) fixes a random permutation π over its edges. Then for any vertex v, the data structure returns the degree of vertex v in the subgraph P produced by Algorithm 2 according to a random permutation π. Each query v to the data structure is answered in $\tilde{O}(T(v,\pi))$ time w.h.p. where $T(v,\pi)$ is as defined in Section 5.

The proof of Lemma 6.1 uses standard ideas from the literature [25, 2]. The only modification, essentially, is to show that these algorithms also work for multi-graphs. Let us focus on the specific algorithm proposed in [2, Appendix A]. Given the adjacency list of a graph $G = (V, E)$, it defines gives a procedure LOWEST(v, i) that first draws a random rank $E \to [0, 1]$ on each edge (implicitly), then for any input vertex v and an integer $i \leq \deg_G(v)$, returns a vertex u such that (v, u) is the i-th lowest rank edge incident to v. It is proved in [2] that if the procedure is called for a fix vertex v and all indices i with $1 \leq i \leq r$, then the total running time is $\tilde{O}(r)$. The only difference between the implementation of our algorithm and the one in [2] is that we have multiple copies of a single edge in the original graph. First, we observe that the procedure LOWEST(v, i), in addition to returning the neighbor u, can also return the rank of the edge (v, u). (This is explicitly computed by LOWEST(v, i) in [2].) Now let G' be the multigraph with K copies of each edge of G. Instead of a multigraph, we can assume that we have K copies of same graph G called $G_1, G_2, \ldots, G_K$. Also, for each i, let LOWEST$_{G_i}$ be the LOWEST procedure corresponding to graph G_i. For each vertex v, we use a balanced binary search tree (BST) that stores the ranks of the lowest incident edge to v in each graph. So at any point during the course of the algorithm, there are at most K different values in the BST of vertex v. Now for the next LOWEST query to the multigraph graph G' for vertex v, we can return the minimum edge in the BST of vertex v. Since K is a constant and the any query to a BST is answered in $O(\log n)$ time, the total running time will be the same as [2, Appendix A] within a $O(\log n)$-factor.